COLLECTION TOTAL — RÉCONCILIATION · 21 LIVRES

Livre 13

Le genèsis humain
de l'automatisation

Partie A

Pourquoi les humains automatisent :
les origines profondes

Images et couverture par Soléane Bélanger

Léo Michel Bélanger, fondateur d'ORIGINALIUM

> *« Automatiser sans comprendre pourquoi, c'est courir sans savoir vers où. »*

Table des matières — Partie A

Partie A : Pourquoi les humains automatisent — les origines profondes

Introduction du Livre 13 — La question que l'industrie n'a jamais vraiment posée p. 7

• L'anecdote fondatrice : l'oncle MIDI p. 8

• Le positionnement de L13 dans la collection p. 10

• Ce que cette Partie A couvre p. 12

Chapitre 1 — L'automatisation : une pulsion humaine ancestrale p. 15

1.1 L'automatisation avant l'informatique p. 16

1.2 Les 5 motivations profondes derrière l'automatisation p. 20

1.3 Le piège de la Motivation 5 — L'oncle MIDI et la curiosité pure p. 26

1.4 La question fondamentale avant d'automatiser p. 31

1.5 Anecdote terrain — L'éditeur de logiciel ERP comptable p. 35

1.6 Scène DREAM-TEAM — La question de Théo p. 38

Chapitre 2 — L'automatisation dans le SDLC réconcilié : les bons endroits p. 43

2.1 Tout n'est pas automatisable — et tout n'a pas à l'être p. 44

2.2 Ce qui doit être automatisé en FORGE p. 48

2.3 Ce qui ne doit pas être automatisé en SENSE p. 52

2.4 Ce qui ne doit pas être automatisé en CARE p. 56

2.5 Le test exploratoire — ce qui ne sera jamais automatisé p. 59

2.6 Anecdote terrain — La grande entreprise en systèmes embarqués p. 62

2.7 Scène DREAM-TEAM — La carte d'automatisation d'ORIGINALIUM p. 65

Chapitre 3 — Les 4 pièges de l'automatisation mal ciblée p. 70

3.1 Piège 1 : Automatiser le mauvais problème p. 71

3.2 Piège 2 : La dette d'automatisation p. 75

3.3 Piège 3 : L'atrophie des compétences p. 79

3.4 Piège 4 : La fausse sécurité p. 83

3.5 Anecdote terrain — Le studio de développement de jeux vidéo p. 87

Chapitre 4 — ONETOOL : la tentation de l'outil miracle p. 92

4.1 L'histoire de ONETOOL p. 93

4.2 La loi de la dépendance à l'outil p. 97

4.3 Comment évaluer un outil d'automatisation p. 101

4.4 Scène DREAM-TEAM — La tentation ONETOOL p. 105

Conclusion Partie A — L'automatisation comme acte humain délibéré p. 110

INTRODUCTION DU LIVRE 13

La question que l'industrie n'a jamais vraiment posée

Pourquoi automatisons-nous vraiment ?

Il existe, dans le corpus de la Collection ToTAL — Réconciliation, une question que tous les livres précédents ont tourné autour sans jamais la formuler directement. Une question qui plane derrière chaque spécification IF…THEN…, derrière chaque session de test exploratoire, derrière chaque pipeline d'intégration continue, derrière chaque cérémonie de qualimétrie. Une question qui semble évidente — et qui, justement parce qu'elle semble évidente, n'est presque jamais posée.

Cette question, la voici : **pourquoi automatisons-nous ?**

La réponse instinctive est prévisible. On automatise pour aller plus vite. Pour réduire les erreurs. Pour faire plus avec moins. Pour rester compétitif. Pour libérer les équipes des tâches répétitives. Ces réponses sont toutes vraies. Elles sont aussi toutes incomplètes. Et leur incomplétude est à l'origine de certaines des erreurs les plus coûteuses, les plus persistantes et les plus silencieuses de l'industrie du logiciel.

Le Livre 13 est peut-être le livre le plus philosophique de la collection — et peut-être aussi le plus important, précisément parce qu'il ose s'arrêter là où l'industrie ne s'arrête jamais. À l'instant de la décision d'automatiser. Avant même que la question du *comment* ne soit posée, L13 insiste sur le *pourquoi*. Et pas seulement sur le *pourquoi* déclaré, celui que l'on inscrit dans les plans de projet et les présentations aux parties prenantes. Sur le *pourquoi* profond. Celui que l'ingénieur chuchote — souvent sans le savoir lui-même — quand il commence à configurer son premier pipeline automatisé.

L'anecdote fondatrice : l'oncle MIDI

Pour entrer dans ce sujet, commençons par une histoire vraie.

Quelque part dans ce monde existe un ingénieur passionné de musique. Appelons-le simplement l'oncle MIDI — non pas parce qu'il est l'oncle de quelqu'un en particulier, mais parce que cette histoire appartient à chacun d'entre nous. Cet homme a consacré des mois à construire, dans sa cave aménagée, un système MIDI entièrement automatisé. Un système d'une sophistication remarquable. Des séquences programmées avec une précision métronome. Des patterns qui se transforment selon des algorithmes qu'il a lui-même conçus. Des variations

harmoniques générées procéduralement, capables de produire une variété musicale quasi infinie sans jamais se répéter exactement.

Le résultat, sur le plan technique, est impeccable. Chaque note à sa place. Chaque transition parfaitement calculée. Le système fonctionne exactement comme prévu — c'est-à-dire, exactement comme l'oncle MIDI l'a voulu.

Un dimanche après-midi, il invite sa famille à écouter. Ils s'installent dans la cave. La musique commence. Elle est belle. Les enfants hochent la tête. Les adultes sourient. Et puis, après quelques minutes, l'une des personnes présentes — peut-être une nièce, peut-être une belle-sœur, peut-être juste quelqu'un qui ne comprend rien à la technique — pose une question simple :

« C'est beau. Mais c'est toi qui joues ? »

« Non, c'est l'ordinateur. »

Long silence.

Ce silence dure peut-être trois secondes. Peut-être cinq. Mais pour l'oncle MIDI, il dure une éternité. Parce que dans ce silence, quelque chose se passe. Une prise de conscience qui ne peut pas être désapprise. L'oncle MIDI réalise soudainement — avec une clarté qui le traverse comme un courant électrique — qu'il a passé des mois à automatiser la musique. Il a automatisé l'*output*. Le son. La structure. La mélodie. Mais il n'a pas automatisé — il n'aurait pas pu automatiser — la joie de faire de la musique. L'expérience de jouer. Le lien entre le musicien et ses auditeurs. La fierté humble de dire : *« C'est moi. »*

Il a couru très vite. Très loin. Avec une grande habileté. Mais vers où ?

C'est cette question — *que cherche-t-on vraiment à automatiser, et pourquoi ?* — qui est au cœur du Livre 13. Et c'est cette anecdote qui en est le point de départ, parce qu'elle illustre, avec une économie narrative remarquable, la tension fondamentale que ce livre cherche à résoudre.

Le positionnement de L13 dans la collection

La Collection ToTAL — Réconciliation comprend 21 livres. Ils couvrent, ensemble, l'ensemble du cycle de développement logiciel sous l'angle de la réconciliation : réconciliation entre vitesse et qualité, entre technique et humain, entre automatisation et jugement, entre données et sens.

Les livres précédents ont construit des fondations solides. Ils ont posé les 6 phases du SDLC réconcilié — ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER — et les 3 zones qui structurent l'organisation humaine autour de ces phases : SENSE, FORGE et CARE. Ils ont introduit la chaîne de valeur Clarté → Qualité → Confiance → Excellence durable et la grande boucle qui ramène CONFIRMER vers ÉLICITER pour que le système s'améliore continuellement.

L13 est le livre qui pose la question ultime en filigrane de tout cela : dans tout ce qu'on a construit — les spécifications IF…THEN…, la qualimétrie, le BEASTEST (optionnel mais puissant quand il est utilisé à bon escient), les post-mortems, l'investigation des anomalies, l'intégration de l'intelligence artificielle comme assistante — quel est le rôle de l'humain ? Et comment s'assurer qu'il reste au centre ?

Ce n'est pas une question anecdotique. C'est la question structurante de toute la collection. Parce que si l'humain disparaît — si l'automatisation finit par le remplacer plutôt que de le libérer — alors la chaîne Clarté → Qualité → Confiance → Excellence durable se brise. La clarté sans jugement humain est du bruit. La qualité sans intention humaine est de la conformité mécanique. La confiance sans relation humaine est une illusion. L'excellence durable sans développement humain continu est une contradiction dans les termes.

L13 répond à cette question non pas en rejetant l'automatisation — ce serait absurde, et ce livre est le dernier à promouvoir le luddisme — mais en cherchant à comprendre ses origines les plus profondes, ses motivations les plus authentiques, ses zones d'application les plus pertinentes et ses pièges les plus insidieux.

Structure du livre — Les deux parties

Le Livre 13 est divisé en deux parties. La **Partie A** — que vous tenez entre les mains — s'intitule *Pourquoi les humains automatisent : les origines profondes*. Elle est philosophique, historique et analytique. Elle pose les bases conceptuelles sans lesquelles la Partie B ne peut pas avoir de sens.

La **Partie B** — *La mécanique de l'automatisation responsable* — sera le complément opérationnel et technique de cette Partie A. Elle répondra à la question : une fois qu'on a bien compris *pourquoi* on automatise, comment fait-on techniquement et humainement pour bien le faire ?

Ces deux parties sont conçues pour être lues ensemble. La Partie A sans la Partie B serait de la philosophie sans application. La Partie B sans la Partie A serait de la technique sans sagesse.

Ce que cette Partie A couvre

La Partie A est organisée autour de quatre chapitres et d'une conclusion.

Le **Chapitre 1** remonte aux origines les plus profondes de l'automatisation — bien avant l'informatique — pour identifier les cinq motivations humaines fondamentales qui poussent les individus et les équipes à automatiser. Il développe en profondeur l'anecdote de l'oncle MIDI comme illustration de la cinquième motivation — la curiosité pure — et ses risques. Il pose la question centrale que toute équipe doit se poser avant d'automatiser quoi que ce soit.

Le **Chapitre 2** ancre l'automatisation dans le SDLC réconcilié et ses trois zones SENSE, FORGE et CARE. Il cartographie avec précision ce qui doit être automatisé, ce qui peut l'être partiellement (la zone intermédiaire ASSISTER), et ce qui ne doit pas l'être — notamment le test exploratoire, qui est par définition l'activité la plus irréductiblement humaine du cycle de développement.

Le **Chapitre 3** identifie et analyse les quatre pièges classiques de l'automatisation mal ciblée : automatiser le mauvais problème, accumuler une dette d'automatisation

invisible, laisser s'atrophier les compétences humaines essentielles, et développer une fausse sécurité statistique.

Le **Chapitre 4** s'attaque à un phénomène récurrent dans l'industrie : la tentation de l'outil miracle — ce que ce livre appelle ONETOOL. Il analyse la psychologie de la dépendance à l'outil et propose un cadre d'évaluation rigoureux pour toute décision d'adoption technologique.

La **Conclusion de la Partie A** synthétise les apprentissages et prépare le terrain pour la Partie B.

À travers ces quatre chapitres, un fil rouge court : l'automatisation est une pulsion humaine noble. Elle l'a toujours été, depuis le moulin à eau jusqu'au pipeline d'intégration continue. Mais comme toute pulsion noble, elle devient destructrice quand elle n'est pas guidée par la compréhension de ses propres motivations. L13 offre cette compréhension.

Commençons.

Collection ToTAL — Réconciliation · Livre 13 · Partie A · Images et couverture par Soléane Bélanger

---

CHAPITRE 1

L'automatisation : une pulsion humaine ancestrale

Des origines préhistoriques aux pipelines modernes — retour aux racines du désir d'automatiser

---

1.1 L'automatisation avant l'informatique

Il est tentant, en lisant un livre sur l'automatisation dans l'industrie du logiciel, d'imaginer que notre sujet commence quelque part dans les années 1960, peut-être avec les premiers compilateurs, ou dans les années 1980 avec les premiers outils de test automatisés. Cette vision est aussi compréhensible que trompeuse. L'automatisation est beaucoup plus vieille que l'informatique. Elle est probablement aussi vieille que l'humanité consciente.

Pour comprendre *pourquoi* les humains automatisent, il faut d'abord comprendre *depuis combien de temps* ils le font — et ce que cela révèle sur leur nature profonde.

Le moulin à eau : la première grande automatisation

Le moulin à eau, dans ses formes les plus primitives, remonte à l'Antiquité. Son principe est d'une simplicité révolutionnaire : utiliser la force d'un phénomène naturel continu — l'eau qui coule — pour accomplir un travail répétitif qu'un humain devrait autrement effectuer à la main, heure après heure, jour après jour. Moudre le grain. Broyer les olives. Fouler le tissu.

Ce qui est remarquable dans cette invention, c'est que son créateur ne s'est pas dit : *Je vais construire une machine pour remplacer l'humain. »* Il s'est dit quelque chose de beaucoup plus intéressant : *« Cette tâche est épuisante, répétitive, et elle pourrait être faite par quelque chose qui ne se fatigue pas. Ça me libérerait pour faire autre chose. »*

Cette pensée — *libérer l'humain du répétitif pour qu'il puisse faire autre chose* — est la première et la plus noble des motivations de l'automatisation. Et elle court, ininterrompue, de l'Antiquité jusqu'aux pipelines CI/CD les plus modernes.

Le métier à tisser Jacquard : la programmabilité avant la lettre

En 1804, un artisan lyonnais nommé Joseph-Marie Jacquard perfectionne un système de cartes perforées qui permet à un métier à tisser de reproduire automatiquement des motifs complexes. Ce qui aurait demandé à un tisserand expert des années de pratique et une attention soutenue à chaque passe de fil peut maintenant être encodé sur des cartons et reproduit à l'identique, indéfiniment, par une machine.

L'invention de Jacquard est extraordinaire pour au moins deux raisons. La première : elle introduit le concept de *programmabilité* — l'idée qu'on peut encoder une séquence d'opérations et demander à une machine de l'exécuter. C'est, dans un sens très réel, l'ancêtre direct du programme informatique. La deuxième raison, moins souvent évoquée : les tisserands lyonnais ont brisé les machines de Jacquard. Plusieurs fois. Pas parce qu'ils ne comprenaient pas leur utilité théorique, mais parce qu'ils avaient peur que leur savoir-faire — accumulé sur des années, transmis de maître à apprenti — ne soit rendu obsolète.

Cette tension — entre la promesse libératrice de l'automatisation et la crainte bien réelle de l'atrophie des compétences humaines — est aussi vieille que l'automatisation elle-même. Elle est encore au cœur de nos débats aujourd'hui, dans l'industrie du logiciel, à chaque fois qu'une équipe s'interroge sur jusqu'où pousser l'automatisation de ses tests ou de son déploiement.

L'horloge mécanique : la mesure du temps rendue autonome

L'horloge mécanique — dans sa forme élaborée, telle qu'elle se développe en Europe entre les XIIe et XVe siècles — est une automatisation d'un type différent. Elle n'automatise pas un travail physique. Elle automatise une *mesure*. Elle rend autonome la perception du temps, libérant les communautés de la nécessité de désigner un gardien du temps, quelqu'un dont la seule responsabilité était de surveiller le soleil ou le sable qui s'écoule et d'annoncer les heures.

C'est une automatisation cognitive. Et elle ouvre une ère nouvelle : celle où les machines ne remplacent pas seulement des muscles, mais aussi des fonctions de surveillance et de mesure. Un précurseur direct des systèmes de monitoring automatisés, des alertes intelligentes et des tableaux de bord de qualimétrie que nous utilisons aujourd'hui dans le SDLC réconcilié.

La chaîne de montage : l'automatisation du processus

Henry Ford, au début du XXe siècle, ne fait pas qu'automatiser des tâches individuelles — il automatise un *processus*. La chaîne de montage de Highland Park ne demande pas à chaque ouvrier de construire une voiture de A à Z. Elle décompose le processus en étapes élémentaires, optimise chaque étape séparément, et fait

défiler le produit devant des travailleurs spécialisés plutôt que de faire défiler les travailleurs autour du produit.

Le gain de productivité est sans précédent. Mais les conséquences humaines sont également profondes. L'artisan qui construisait une voiture entière avait une relation de bout en bout avec son produit. L'ouvrier qui visse le même boulon dix mille fois par jour a perdu cette relation. Ce qu'on a automatisé ici, c'est non seulement le processus physique, mais aussi le sens du travail artisan.

Cette observation n'est pas un jugement de valeur. C'est une observation factuelle avec des implications pratiques majeures pour l'industrie du logiciel. Quand on décompose le cycle de développement en phases hyper-spécialisées et qu'on automatise chaque phase séparément, on risque de perdre quelque chose d'essentiel : le sens de bout en bout, la vision du produit complet, la fierté du tisserand qui peut dire *« c'est moi qui l'ai fait ».*

C'est pourquoi le SDLC réconcilié insiste sur les 3 zones — SENSE, FORGE, CARE — non pas comme des silos étanches, mais comme des zones de responsabilité *partagée*, toutes orientées vers le même produit, toutes animées par les mêmes valeurs de clarté, de qualité et de confiance.

L'informatique : l'accélération, pas la rupture

Quand l'informatique arrive — avec ses premiers compilateurs, ses systèmes batch, puis ses architectures client-serveur et enfin le cloud — elle n'invente pas l'automatisation. Elle l'*accélère* et l'*étend* à des domaines qui n'avaient jamais été accessibles. Elle rend possible l'automatisation de processus cognitifs complexes, de décisions, de prédictions. Mais elle ne change pas la nature fondamentale de la pulsion qui pousse les humains à automatiser.

C'est ce que ce chapitre cherche à établir comme fondation : l'automatisation n'est pas une invention de notre époque. C'est une expression constante de la nature humaine. Pour bien l'utiliser dans le contexte du développement logiciel, il faut comprendre d'où elle vient — et ce qu'elle cherche vraiment à accomplir.

> *L'automatisation est aussi vieille que la conscience humaine. Chaque époque produit sa version de la même pulsion : libérer l'humain du répétitif pour qu'il puisse faire ce que seul lui peut faire.*

1.2 Les 5 motivations profondes derrière l'automatisation

Si l'automatisation est universelle, si elle traverse toutes les civilisations et toutes les industries, c'est parce qu'elle répond à des besoins humains fondamentaux. Après des années d'observation terrain dans des contextes organisationnels très variés — startups en hypercroissance, grandes institutions financières coopératives, studios de jeux vidéo, entreprises de traçabilité industrielle, firmes conseil nationales en transformation numérique — cinq motivations profondes se dégagent.

Ces cinq motivations sont toutes légitimes. Elles sont toutes, dans les bonnes conditions, de puissants moteurs d'excellence. Et elles sont toutes, dans les mauvaises conditions, capables de produire exactement l'inverse de ce qu'elles promettent.

Motivation 1

LA LIBÉRATION

Se libérer des tâches répétitives pour faire des choses plus intéressantes — et plus importantes.

C'est la motivation la plus ancienne, la plus universelle, et dans un sens, la plus noble. Le moulin à eau était animé par cette motivation. Le tisserand qui configure le métier Jacquard est animé par cette motivation. L'équipe QA qui automatise ses tests de régression pour se concentrer sur le test exploratoire est animée par cette motivation.

La Motivation 1 est fondamentalement altruiste dans son orientation : elle cherche à libérer du temps humain vers des activités à plus haute valeur ajoutée. Mais elle a un prérequis implicite souvent oublié : il faut qu'il y ait réellement quelque chose de plus important et de plus intéressant vers quoi ce temps sera redirigé. Si l'automatisation libère du temps vers… rien de particulier, ou vers d'autres tâches tout aussi répétitives et automatisables, alors la libération est illusoire.

La question à poser systématiquement quand la Motivation 1 est invoquée : *« Vers quoi exactement est-ce que cette automatisation libère du temps ? »* Si la réponse est vague ou absente, l'automatisation est prématurée.

Motivation 2

LA PRÉCISION

Atteindre un niveau de précision impossible à maintenir pour l'humain sur la durée.

L'humain est extraordinairement précis — pendant un certain temps. Sur la durée, il se fatigue, se distrait, s'ennuie. Il introduit des variations involontaires. Sa précision dépend de son état de la journée, de son niveau de stress, de son engagement dans la tâche. Ces variations sont naturelles, humaines, et souvent inévitables.

Certaines tâches requièrent une précision absolue et répétable que seule une machine peut garantir. Les tests de régression en sont le meilleur exemple : exécuter exactement les mêmes 500 vérifications, dans exactement le même ordre, avec exactement les mêmes données, et comparer exactement les mêmes résultats — c'est une tâche pour laquelle l'automatisation est non seulement supérieure, mais irremplaçable.

La Motivation 2 est particulièrement forte dans les contextes à haute criticité : systèmes embarqués temps réel, logiciels médicaux, systèmes financiers, infrastructures de sécurité. Dans ces domaines, l'automatisation de la vérification n'est pas un luxe — c'est une nécessité.

Motivation 3

L'ÉCHELLE

Faire à 1 000, à 100 000, à 1 000 000 ce qui ne peut se faire qu'à 1 à la main.

Un humain peut rédiger une facture. Dix humains peuvent en rédiger dix. Mais rédiger un million de factures personnalisées, en temps réel, en s'adaptant à des centaines de variables différentes — c'est hors de portée de toute équipe humaine, aussi grande soit-elle.

La Motivation 3 est celle qui caractérise le mieux l'ère numérique. L'automatisation à l'échelle a rendu possible des modèles économiques entiers : le commerce électronique, le streaming personnalisé, les services financiers numériques. Sans elle, le monde numérique tel qu'on le connaît n'existerait tout simplement pas.

Dans le contexte du développement logiciel, la Motivation 3 se traduit par l'automatisation du déploiement continu, du monitoring distribué, de la gestion des incidents à l'échelle. Elle est légitime et puissante. Son danger principal : créer des systèmes tellement complexes et tellement automatisés qu'aucun humain ne peut plus en comprendre le comportement global.

Motivation 4

LA SÉCURITÉ

Éliminer les tâches dangereuses, épuisantes ou psychologiquement délétères pour les humains.

Les robots qui manipulent des substances toxiques, les systèmes automatisés qui surveillent les processus nucléaires, les algorithmes qui filtrent les contenus violents avant qu'un modérateur humain ne les voie — tous ces systèmes sont animés par la Motivation 4. Ils cherchent à protéger l'humain.

Dans l'industrie du logiciel, la Motivation 4 se manifeste différemment. Elle justifie l'automatisation des déploiements nocturnes pour que personne n'ait à travailler à 3h du matin. Elle justifie l'automatisation des alertes de sécurité pour que les équipes ne soient pas submergées par des centaines de faux positifs. Elle justifie l'automatisation de la surveillance de la dette technique pour éviter le burn-out des équipes qui se retrouvent à gérer des crises perpétuelles.

La Motivation 4 est peut-être la plus vertueuse de toutes — et la moins souvent explicitement invoquée. Pourtant, l'automatisation qui protège les humains de l'épuisement, de la pression nocturne et du bruit opérationnel est l'une des plus précieuses contributions que la technologie peut apporter à une organisation saine.

Motivation 5

LA CURIOSITÉ

Parce que c'est techniquement fascinant — l'ingénieur qui automatise pour le plaisir d'automatiser.

Voilà la motivation dont personne ne parle dans les présentations aux comités de direction, mais que tout ingénieur honnête reconnaîtra immédiatement. La curiosité pure. Le *« je peux donc je dois »*. L'excitation intellectuelle de construire quelque chose d'élégant et de fonctionnel. La satisfaction esthétique d'un pipeline parfaitement configuré, d'un script qui roule sans accroc, d'un système qui s'auto-corrige sans intervention humaine.

Cette motivation est humaine, profonde et légitime. Elle est à la source de nombreuses des innovations technologiques les plus importantes de l'histoire. Elle est aussi, quand elle n'est pas questionnée et encadrée, la plus dangereuse de toutes.

C'est la Motivation 5 que l'oncle MIDI a suivie. Et c'est la Motivation 5 que ce chapitre va maintenant examiner en profondeur.

1.3 Le piège de la Motivation 5 — L'oncle MIDI et la curiosité pure

L'oncle MIDI n'est pas un mauvais ingénieur. Loin de là. Il est probablement — ce l'est presque toujours dans cette configuration — un ingénieur brillant, passionné, méticuleux. Le système qu'il a construit dans sa cave est techniquement remarquable. La programmation des séquences MIDI, la gestion des patterns, les algorithmes de variation harmonique — tout cela témoigne d'une compétence réelle et d'une intelligence créatrice indéniable.

Le problème n'est pas sa compétence. Le problème est la question qu'il ne s'est jamais posée pendant les mois où il construisait son système. Pas une seule fois.

Cette question, la voici : *Qu'est-ce que j'automatise, et pourquoi ?*

La réponse évidente aurait été : *j'automatise la production de musique pour me libérer du besoin de jouer d'un instrument.* Mais si on pousse un peu : libéré du besoin de jouer, pour quoi faire ? Pour écouter la musique de son propre système ? Pour partager la musique avec sa famille ? Pour avoir un fond sonore permanent dans la maison ?

Aucune de ces réponses n'est absurde. Elles sont toutes valides, en fonction des objectifs. Mais le problème de l'oncle MIDI est qu'il n'a pas vérifié que sa réponse correspondait à ce que lui et ses proches valorisaient vraiment dans la musique.

Car ce que la famille valorisait dans la musique — ce qui leur manquait depuis que l'oncle avait arrêté de jouer pour construire son système — c'était précisément ce que le système avait éliminé. La présence. L'imperfection chaleureuse. La note légèrement ratée corrigée avec un sourire. La décision improvisée de rejouer un passage parce que quelqu'un dans la pièce l'avait apprécié. Le regard partagé entre le musicien et ses auditeurs.

Ce n'est pas la musique que l'oncle MIDI a automatisée. C'est la relation musicale. Et cette relation-là ne s'automatise pas.

Le phénomène « je peux donc je dois »

Ce phénomène — *je peux donc je dois* — est l'une des forces les plus puissantes et les plus mal comprises de l'industrie du logiciel. Il s'active dès qu'un ingénieur ou une équipe réalise qu'une automatisation est techniquement faisable. Le fait qu'elle soit *faisable* crée immédiatement une pression implicite pour qu'elle soit *faite*. Ne pas l'automatiser alors qu'on en serait capable est perçu — consciemment ou non — comme un manque de rigueur, voire une forme de paresse intellectuelle.

Ce phénomène est amplifié par plusieurs dynamiques organisationnelles typiques :

- **La culture de la sophistication technique :** dans les équipes de développement, l'automatisation complexe confère un statut. L'ingénieur qui a monté un pipeline CI/CD avec du déploiement canary, des feature flags dynamiques et des rollbacks automatiques est admiré. Son travail est visible, impressionnant, discutable dans les conférences techniques.
- **L'invisibilité de la non-automatisation :** à l'inverse, la décision délibérée de ne pas automatiser quelque chose — parce qu'on a évalué que l'automatisation n'apporterait pas de valeur, ou qu'elle éroderrait une compétence essentielle — est invisible. On ne peut pas la montrer. On ne peut pas s'en vanter. Elle passe souvent pour de l'inaction.
- **La dérive de la définition de « done » :** dans de nombreuses équipes, une feature ou une infrastructure n'est considérée « done » que quand elle est automatisée. Ce qui crée une pression pour automatiser même quand l'automatisation n'est pas la meilleure réponse.
- **L'ennui structurel :** certaines activités sont simplement ennuyeuses. Documenter les cas de test manuels est ennuyeux. Écrire des scripts d'automatisation est intéressant. Cette asymétrie émotionnelle — réelle et compréhensible — pousse naturellement vers l'automatisation, indépendamment de sa valeur relative.

Quand la curiosité crée de la valeur — et quand elle en détruit

Il serait injuste — et inexact — de présenter la curiosité pure comme une force toujours néfaste. L'histoire de la technologie est pleine d'automatisations qui ont démarré par curiosité pure et qui ont fini par changer le monde. Les premiers compilateurs ont été construits par des informaticiens curieux de voir si on pouvait traduire automatiquement du code symbolique en instructions machine. Les premiers systèmes de contrôle de version ont été construits par des développeurs fatigués de gérer manuellement des copies de fichiers. La première intégration continue a été mise en place par une équipe qui voulait voir si c'était possible de builder automatiquement à chaque commit.

La curiosité crée de la valeur quand elle est suivie d'une question : *Et maintenant, est-ce que ce que j'ai construit répond à un vrai besoin ?*

Elle en détruit quand elle reste dans sa propre logique, déconnectée des besoins réels, et quand l'output de l'automatisation est valorisé pour lui-même plutôt que pour l'effet qu'il produit sur les personnes et les processus qu'il est censé servir.

L'oncle MIDI n'a jamais posé la question. Il était trop profondément engagé dans la fascination de construire. Et c'est en cela qu'il est représentatif de l'industrie : non pas parce qu'il est incompétent, mais précisément parce qu'il est compétent et passionné, et que cette compétence et cette passion l'ont empêché de s'arrêter pour demander *pourquoi*.

L'oncle MIDI dans l'industrie du logiciel

Dans l'industrie du logiciel, l'oncle MIDI se manifeste sous des formes très concrètes :

- L'équipe qui construit un framework de test automatisé maison pendant six mois, avec des dizaines de milliers de lignes de code d'infrastructure, pour finalement réaliser qu'un outil existant aurait répondu au besoin en une semaine.
- L'architecte qui déploie un pipeline de déploiement continu à 17 étapes, avec des validations automatiques à chaque étape, pour un projet dont les releases sont mensuelles et dont l'équipe compte 3 développeurs.

- L'équipe QA qui automatise 2 000 scénarios de test en six mois — des scénarios qui ne testent pas les véritables risques du produit — pendant que les bugs critiques continuent de s'introduire par des chemins que personne n'a pensé à couvrir.
- L'ingénieur DevOps qui configure un système de monitoring avec 400 métriques différentes, dont 380 ne sont jamais regardées et dont les 20 qui comptent sont noyées dans le bruit.

Dans tous ces cas, la motivation était la curiosité. La technique était impressionnante. Et le résultat — mesuré à l'aune du vrai besoin, pas de la sophistication de la solution — était décevant.

La règle de la curiosité encadrée

Le SDLC réconcilié ne cherche pas à étouffer la curiosité. Il cherche à l'encadrer. La règle est simple : la curiosité est un excellent *détonateur* de l'automatisation — elle permet d'imaginer des solutions que personne n'aurait pensé à envisager — mais un mauvais *gouverneur*. Elle doit être suivie, dans les 48 à 72 heures d'un nouveau projet d'automatisation, d'une question de gouvernance :

Ce que je construis — vers quoi est-ce que ça libère de la capacité humaine, et est-ce que cette libération correspond à un vrai besoin identifié par les parties prenantes du SDLC réconcilié ?

Si la réponse à cette question est positive et précise, la curiosité continue à alimenter le projet. Si la réponse est vague ou absente, un signal d'arrêt doit être activé — pas pour abandonner l'idée, mais pour la soumettre à un processus de validation plus rigoureux avant d'investir davantage.

1.4 La question fondamentale avant d'automatiser

On en arrive maintenant au cœur de ce chapitre — et, d'une certaine manière, au cœur de tout le Livre 13.

Avant d'automatiser quoi que ce soit, une question doit être posée et, surtout, répondue avec précision. Cette question n'est pas technique. Elle n'est pas organisationnelle. Elle est fondamentalement humaine.

> *Qu'est-ce que cette automatisation libère, et vers quoi ?*

Cette question a deux composantes également importantes. La première est : *qu'est-ce que cette automatisation libère ?* Du temps ? De l'attention ? De l'énergie cognitive ? Des compétences ? La deuxième composante est : *vers quoi ?* Vers quelles activités, quelles responsabilités, quels défis cette capacité libérée va-t-elle se rediriger ?

La réponse doit être *précise*. « Ça libère du temps vers des choses plus importantes » n'est pas une réponse précise. « Ça libère les QA de l'exécution des tests de régression pour qu'ils puissent consacrer deux fois plus de temps au test exploratoire des nouvelles fonctionnalités » est une réponse précise. « Ça libère l'équipe de déploiement des lancements nocturnes manuels pour qu'ils puissent se concentrer sur l'architecture de la plateforme pendant les heures ouvrables » est une réponse précise.

La précision importe parce qu'elle permet une évaluation rétrospective. Si, six mois après la mise en place de l'automatisation, le temps libéré n'a pas été redirigé vers l'activité promise, c'est un signal fort que quelque chose ne fonctionne pas — soit dans l'automatisation elle-même, soit dans la gouvernance de l'organisation.

Le test des trois réponses

Dans la pratique du SDLC réconcilié, une équipe qui envisage une automatisation significative est invitée à formuler au moins trois réponses distinctes à la question *« vers quoi est-ce que ça libère ? »* et à évaluer leur qualité.

Type de réponse	Exemple	Verdict
Réponse vague	« Ça va nous permettre d'être plus efficaces. »	**Signal d'arrêt**

Type de réponse	Exemple	Verdict
Réponse générique	« Ça libère du temps pour des tâches à plus haute valeur ajoutée. »	**À approfondir**
Réponse précise	« Ça libère Karim et son équipe de 4h de tests manuels par sprint, qu'ils pourront consacrer à l'exploration des scénarios d'erreur des nouvelles fonctionnalités de paiement. »	**Aller de l'avant**

Quand la réponse est « vers rien de particulier »

Une réponse honnête qui mérite d'être mentionnée : *« Cette automatisation libère du temps vers... rien de particulier. »* C'est-à-dire : l'équipe est sous-chargée, ou l'activité automatisée est marginale, ou le gain de temps sera si faible qu'il sera absorbé par le bruit quotidien sans jamais se matérialiser en capacité réelle.

Quand cette réponse émerge honnêtement, l'automatisation n'est pas nécessairement à rejeter — mais elle demande une réévaluation. Peut-être que le problème à résoudre n'est pas un manque de temps, mais un manque de priorisation, de clarté sur les objectifs, ou d'alignement entre les capacités de l'équipe et les besoins réels du produit. Dans ce cas, automatiser ne résoudra rien — ce n'est pas le bon outil pour le bon problème.

Quand la réponse est incertaine

Il existe aussi une réponse légitime qui est : *« Je ne sais pas encore exactement vers quoi ça libère, mais je pense que ça libèrerait du potentiel si on l'essayait. »* Cette réponse, dans le cadre du SDLC réconcilié, est acceptable — à condition qu'elle soit accompagnée d'un dispositif de validation. On autorise une période d'expérimentation courte et délimitée, avec des critères de succès définis à l'avance, après laquelle on évalue si le potentiel pressenti s'est matérialisé.

C'est la différence entre explorer intelligemment et dériver sans boussole. L'oncle MIDI n'a pas exploré intelligemment. Il a dérivé — très élégamment, très compétamment — mais sans boussole.

1.5 Anecdote terrain — L'éditeur de logiciel ERP comptable

Anecdote terrain · Éditeur de logiciel ERP comptable

Les noms, dates et données spécifiques ont été rendus anonymes conformément aux règles de la Collection ToTAL.

Un éditeur de logiciel ERP comptable bien établi, opérant dans un marché de niche avec une clientèle de PME fidèles, a entrepris un ambitieux programme d'automatisation de ses tests. L'objectif déclaré était noble : libérer l'équipe QA des tests de régression répétitifs pour qu'elle puisse se concentrer sur la validation des nouvelles fonctionnalités du produit, qui était en pleine expansion.

Le programme a été un succès technique remarquable. En l'espace de dix-huit mois, l'équipe a automatisé 80% de ses cas de test. Le framework d'automatisation était propre, bien documenté, intégré au pipeline CI/CD. Les rapports de couverture affichaient des chiffres impressionnants. Lors des revues trimestrielles, les tableaux de bord présentaient des métriques enviables.

Le problème, qui n'a émergé que progressivement, puis soudainement, était le suivant : les 80% de tests automatisés couvraient des scénarios qui ne cassaient jamais. Des chemins heureux bien balisés, des calculs arithmétiques basiques, des validations de champs standards. Ces scénarios avaient été choisis pour leur facilité d'automatisation, pas pour leur importance dans la couverture des risques réels du produit.

Les 20% restants — les scénarios qui n'avaient pas été automatisés parce qu'ils étaient trop complexes, trop contextuels, trop dépendants de l'expertise métier — n'étaient jamais exécutés non plus. L'équipe QA, maintenant libérée des tests de régression, avait été réaffectée à d'autres projets. Personne n'avait vraiment pensé à s'assurer que les 20% non automatisés continueraient d'être testés manuellement.

Le résultat : lors d'une mise à jour majeure du moteur de calcul comptable, un bug critique passa en production. Ce bug affectait un scénario très spécifique : le calcul des taxes sur les transactions multi-devises impliquant une note de crédit partielle sur une facture déjà partiellement payée. Un scénario marginal en fréquence, mais catastrophique quand il survient — et il survenait pour environ 3% des clients, tous les mois.

L'investigation post-incident révéla que ce scénario avait été explicitement identifié comme « complexe à automatiser » dans les notes du programme d'automatisation. Il avait été reporté. Puis oublié. Puis... plus personne ne savait qu'il existait.

L'équipe avait automatisé 80% de ses tests. La couverture sur le papier était impressionnante. Et pourtant, un bug critique qui aurait été détecté immédiatement par n'importe quel testeur humain connaissant le domaine était passé à travers toutes les mailles du filet automatisé.

La leçon n'est pas que l'automatisation était mauvaise. La leçon est que l'équipe avait automatisé *ce qui était facile à automatiser*, pas *ce qui était important à tester*. Elle avait suivi la pente naturelle de la Motivation 5 — l'ingénierie pour le plaisir de l'ingénierie — sans jamais poser la question fondamentale : *Qu'est-ce que cette automatisation libère, et vers quoi ?*

La réponse honnête aurait été : *« Cette automatisation libère du temps vers... des scénarios complexes qui demandent une expertise métier profonde. »* Et la conséquence logique de cette réponse aurait été de s'assurer que ces scénarios complexes étaient effectivement pris en charge — soit par automatisation ciblée (plus difficile mais possible), soit par des sessions de test exploratoire structurées.

Ni l'une ni l'autre n'avait été faite.

1.6 Scène DREAM-TEAM — La question de Théo

Scène DREAM-TEAM · ORIGINALIUM

Équipe : Rania (Tech Lead) · Théo (PO) · Karim (QA Lead) · ORIGINALIUM

Jeudi matin, neuf heures moins dix. La salle de réunion d'ORIGINALIUM est encore à moitié endormie. Karim est arrivé le premier, comme toujours, et a déjà couvert le tableau blanc de trois colonnes denses. Il a préparé une présentation sur le programme d'automatisation du prochain trimestre. C'est visiblement un travail de plusieurs soirées.

Rania arrive avec son café, jette un coup d'œil au tableau, hoche la tête avec l'air de quelqu'un qui reconnaît le travail bien fait. Théo arrive en dernier, légèrement essoufflé, glisse son ordinateur sur la table et demande : « On commence ? »

Karim se lève, prend le marqueur. « Alors voilà ce que j'ai préparé. Pour le prochain trimestre, l'objectif est de monter notre couverture d'automatisation à 75%. J'ai listé les 23 modules à couvrir en priorité, avec les estimations d'effort pour chaque. J'ai aussi identifié trois frameworks candidats pour la nouvelle couche d'automatisation de l'API, avec une comparaison technique — »

Théo lève la main doucement. Pas de manière agressive. Plutôt comme quelqu'un qui veut juste s'assurer d'avoir bien entendu. « Karim, avant de continuer — est-ce que tu peux me dire pourquoi ? »

Karim s'arrête. « Pourquoi quoi ? »

« Pourquoi les 23 modules. Pourquoi 75%. Avant de dire QUOI, dis-moi POURQUOI. Qu'est-ce que cette automatisation va libérer ? »

Silence. Karim regarde le tableau. La question est simple. Et pourtant, il réalise qu'il n'a pas de réponse précise prête. Il a une réponse générique — « ça va accélérer les cycles de validation, ça va réduire les erreurs humaines dans les régressions » — mais rien de précis sur *vers quoi* le temps libéré sera redirigé.

Rania est observatrice. Elle laisse le silence durer une seconde de plus, puis dit tranquillement : « Karim, qu'est-ce que tu *veux* faire que tu ne peux pas faire en ce moment ? »

Karim pose le marqueur. Il s'assied. Et après un moment, il dit quelque chose d'une honnêteté remarquable pour une réunion d'équipe : « Je pense… je pense que j'automatise parce que c'est plus intéressant à faire que de documenter les cas manuels. L'automatisation, c'est un problème d'ingénierie. C'est stimulant. La documentation des cas manuels, c'est du travail de fourmi. Et honnêtement, si je suis honnête avec moi-même, je ne me suis pas demandé si les 23 modules que j'ai listés sont les bons 23. J'ai listé ceux que je savais automatiser. »

Théo sourit. Pas d'un sourire condescendant — d'un sourire de quelqu'un qui reconnaît quelque chose d'important. « C'est exactement la bonne réponse. La vraie. Pas celle qu'on met dans les slides. »

Ce qui suit dans la réunion est peut-être la conversation la plus productive que l'équipe a eue depuis des mois. Ils reprennent la liste des 23 modules, cette fois en commençant par une question différente : *quels sont les risques les plus importants que nous voulons couvrir, et parmi ceux-là, lesquels bénéficieraient vraiment de l'automatisation ?* La réponse réduit la liste à 11 modules — mais 11 modules vraiment importants, dont l'automatisation libèrerait Karim et son équipe pour consacrer le reste du temps sprint à des sessions d'exploration structurées sur les fonctionnalités nouvelles.

La décision de Théo d'interrompre Karim avec un simple « pourquoi ? » a changé la direction de tout un programme d'automatisation.

C'est la puissance de la question fondamentale.

À retenir — Chapitre 1

- L'automatisation est une pulsion humaine ancestrale — elle précède l'informatique de plusieurs millénaires. Sa noblesse originelle est de libérer

l'humain du répétitif pour qu'il puisse se consacrer à ce que seul il peut faire.

- Les 5 motivations profondes — Libération, Précision, Échelle, Sécurité, Curiosité — sont toutes légitimes. Toutes peuvent devenir des pièges quand elles ne sont pas questionnées.
- La Motivation 5 (Curiosité pure) est la plus répandue dans l'industrie du logiciel — et la plus dangereuse quand elle n'est pas encadrée par la question du *pourquoi*.
- La question fondamentale avant toute automatisation : *Qu'est-ce que cette automatisation libère, et vers quoi ?* La réponse doit être précise et vérifiable.
- L'oncle MIDI n'a pas automatisé la musique. Il a automatisé l'output musical — et en faisant cela, il a éliminé la relation musicale, qui était ce que sa famille valorisait vraiment.
- Automatiser ce qui est facile à automatiser plutôt que ce qui est important à automatiser est l'erreur la plus commune — et la plus coûteuse — de l'industrie.

Symptômes à observer

- L'équipe parle de couverture d'automatisation en pourcentage sans pouvoir dire ce que ce pourcentage représente en termes de risques couverts.
- Les sujets d'automatisation sont choisis en fonction de leur faisabilité technique, non de leur importance pour la qualité du produit.
- Personne ne sait clairement vers quoi le temps libéré par l'automatisation est redirigé — et personne ne le mesure.
- L'enthousiasme pour les nouveaux outils d'automatisation dépasse systématiquement l'enthousiasme pour les résultats qu'ils produisent.

- Les équipes automatisent des tests qui ne cassent jamais, pendant que des domaines à haute criticité restent non couverts.
- L'automatisation est perçue comme une fin en soi, et non comme un moyen d'atteindre une fin définie.

Questions de réflexion

1. Dans votre équipe, si vous deviez répondre honnêtement à la question « vers quoi nos automatisations actuelles libèrent-elles du temps ? », quelle serait votre réponse précise ?
2. Y a-t-il des automatisations en place dans votre organisation dont vous ne savez pas vraiment pourquoi elles ont été créées ? Que révèle cette ignorance ?
3. La prochaine fois que vous envisagez d'automatiser quelque chose, pouvez-vous identifier quelle(s) des 5 motivations vous animez — et si celle(s)-ci est (sont) bien calibrée(s) ?
4. Qu'est-ce que l'oncle MIDI aurait dû se demander avant de commencer son système MIDI ? Et comment vous assurez-vous de vous poser ces questions dans votre propre travail ?
5. Y a-t-il des choses dans votre environnement de travail actuel qui ont été automatisées et qui auraient peut-être mieux servi leur objectif si elles étaient restées humaines ?

Signal de vigilance

Si un programme d'automatisation démarre sans qu'on ait répondu

précisément et explicitement à la question « vers quoi est-ce que ça libère ? », arrêtez-vous. Pas pour toujours — mais le temps de répondre honnêtement à cette question. Un programme d'automatisation sans réponse précise à cette question est un programme qui risque de produire de la sophistication technique sans valeur réelle, et possiblement de détruire des compétences humaines essentielles en cours de route.

Ce n'est pas de la prudence excessive. C'est de la gouvernance responsable de la capacité humaine.

Collection ToTAL — Réconciliation · Livre 13 · Chapitre 1 · Images et couverture par Soléane Bélanger

---

CHAPITRE 2

L'automatisation dans le SDLC réconcilié : les bons endroits

Cartographier avec précision ce qui doit être automatisé, assisté, ou laissé entièrement humain

---

2.1 Tout n'est pas automatisable — et tout n'a pas à l'être

Il existe, dans les cultures techniques les plus avancées, une croyance implicite qui mérite d'être nommée et questionnée : la croyance que l'automatisation est, par définition, supérieure à l'activité humaine. Que si quelque chose peut être automatisé, il devrait l'être. Que l'humain est, au fond, une solution de repli pour les tâches que les machines ne savent pas encore faire — une solution temporaire, en attente de la prochaine percée technologique.

Cette croyance est fausse. Et elle est dangereuse.

Certaines activités sont objectivement mieux réalisées par des machines. Certaines sont objectivement mieux réalisées par des humains. Et une troisième catégorie — peut-être la plus intéressante et la plus importante — est mieux réalisée par des humains *assistés* de machines.

Le SDLC réconcilié, avec ses 6 phases (ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER) et ses 3 zones (SENSE · FORGE · CARE), offre un cadre structuré pour cartographier ces trois catégories. Ce chapitre propose cette cartographie — non pas comme une vérité absolue et immuable, mais comme un point de départ pour une réflexion d'équipe structurée.

La grille d'automatisation du SDLC réconcilié

Zone	Phase	Activités clés	Posture recommandée
SENSE	ÉLICITER	Entretiens utilisateurs, ateliers de besoins, observation terrain, empathie, formulation d'hypothèses	**HUMAIN UNIQUEMENT**
SENSE	DÉFINIR	Rédaction des spécifications IF… THEN…, priorisation des exigences, décision de périmètre	**HUMAIN + ASSISTANCE**
FORGE	PLANIFIER	Estimation d'effort, planning de sprint, identification des dépendances, stratégie de test	**HUMAIN + ASSISTANCE**
FORGE	IMPLÉMENTER	Développement, tests unitaires, intégration	**AUTOMATISER PRIORITAIREMENT**

Zone	Phase	Activités clés	Posture recommandée
		continue, déploiement continu, analyse statique, tests de régression	
CARE	DIFFUSER	Communication des releases, formation des utilisateurs, documentation contextuelle, gestion du changement	**HUMAIN UNIQUEMENT**
CARE	CONFIRMER	Qualimétrie, analyse des métriques, décision Go/No-Go, post-mortem, boucle de retour ÉLICITER	**DONNÉES AUTO · DÉCISION HUMAINE**

Ce tableau est une simplification — délibérée. La réalité de chaque organisation sera plus nuancée, avec des zones d'automatisation qui se déplacent en fonction des contextes, des maturités, des contraintes réglementaires et des cultures d'équipe. Mais ce tableau offre une orientation claire : l'automatisation est principalement pertinente en FORGE, dans la phase IMPLÉMENTER. Elle doit être utilisée avec discernement et comme assistance en SENSE et en CARE. Et certaines activités humaines essentielles — l'élicitation, la relation utilisateur, la décision de Go/No-Go, le post-mortem — ne doivent tout simplement pas être automatisées.

La zone intermédiaire : ASSISTER

Ce tableau introduit une catégorie qui mérite d'être développée : la posture HUMAIN + ASSISTANCE. Dans la pratique, c'est la zone la plus répandue — et la moins bien adressée.

ASSISTER signifie : une machine effectue un travail préparatoire, analytique ou récapitulatif, et un humain exerce son jugement sur ce que la machine a produit. La

machine ne décide pas. Elle informe. Elle prépare. Elle structure. Elle suggère. Mais la décision finale — et la responsabilité qui l'accompagne — reste humaine.

Exemples concrets :

- Un outil de revue de code automatisé qui identifie les violations de style et les problèmes de complexité cyclomatique — et un développeur senior qui décide lesquels méritent une correction immédiate et lesquels peuvent attendre.
- Un algorithme de priorisation qui analyse l'historique des bugs, la fréquence d'utilisation des fonctionnalités et les feedbacks utilisateurs pour suggérer un ordre de priorité pour le backlog — et un PO qui valide, ajuste et assume la décision finale.
- Un système de monitoring qui agrège les métriques de qualimétrie et génère un rapport de situation — et un comité humain qui interprète ces métriques dans leur contexte organisationnel et décide des actions.

La posture ASSISTER est peut-être la contribution la plus précieuse que l'automatisation peut apporter à des activités complexes : non pas remplacer le jugement humain, mais l'alimenter avec des informations plus riches, plus complètes et plus rapides que ce que l'humain pourrait assembler seul.

2.2 Ce qui doit être automatisé en FORGE

La zone FORGE — centrée sur la phase IMPLÉMENTER — est l'espace naturel de l'automatisation dans le SDLC réconcilié. C'est ici que l'automatisation apporte sa valeur la plus évidente et la plus mesurable. Et c'est ici que ne pas automatiser est, dans la plupart des contextes, une erreur de gouvernance.

Tests unitaires et SMALL

Les tests unitaires — appelés SMALL dans le vocabulaire du SDLC réconcilié pour rappeler leur caractéristique fondamentale : petits, rapides, focalisés, sans dépendances externes — sont l'exemple canonique d'automatisation bien ciblée. Chaque test unitaire vérifie une fonction, une méthode, un comportement atomique.

Il s'exécute en quelques millisecondes. Il peut être relancé à chaque commit, à chaque sauvegarde de fichier, à chaque modification de code.

Cette automatisation libère les développeurs de la nécessité de vérifier manuellement le comportement de chaque fonction à chaque modification. Ce faisant, elle leur permet de changer le code avec confiance, de refactoriser sans crainte, d'innover sans anxiété. Ce qu'elle libère — la confiance dans le code — est précieux et bien défini. C'est une automatisation exemplaire.

Intégration continue

L'intégration continue (CI) automatise le processus de vérification que le code de tous les membres de l'équipe, combiné ensemble, continue de fonctionner correctement. Sans CI, cette vérification repose sur la discipline individuelle et la chance. Avec CI, elle est systématique, immédiate et exhaustive.

La CI libère les équipes d'une source de stress chronique : le syndrome de la « merge de fin de sprint », où des semaines de développement parallèle se heurtent soudainement à des conflits et des incompatibilités que personne ne savait exister. Cette automatisation libère du temps et de l'énergie vers ce qui compte : le développement de nouvelles fonctionnalités et l'amélioration de la qualité.

Déploiement continu

Le déploiement continu (CD) automatise le processus de mise en production. Il réduit le risque associé à chaque déploiement — car des déploiements fréquents et automatisés sont statiquement moins risqués que des déploiements rares et manuels — et il libère les équipes des rituels anxiogènes de la release nocturne.

Un déploiement automatisé bien construit inclut des validations progressives, des mécanismes de rollback automatique en cas d'anomalie, et des alertes qui permettent une détection précoce des problèmes. Il libère les équipes de surveillance nocturne — ce qui est, rappelons-le, une application directe de la Motivation 4 (la Sécurité).

Analyse statique et sécurité

L'analyse statique — vérification automatique de la qualité syntaxique et sémantique du code sans l'exécuter — détecte des classes entières d'erreurs avant même que le code ne soit testé. Les vulnérabilités de sécurité communes, les fuites mémoire potentielles, les violations de standards de codage : autant d'éléments que l'analyse statique peut identifier systématiquement, libérant les revues de code humaines pour se concentrer sur la logique métier, l'architecture et les décisions de design.

Tests de régression automatisés

La régression — vérifier que ce qui fonctionnait avant continue de fonctionner après une modification — est le cas d'usage le plus classique de l'automatisation en FORGE. C'est une tâche répétitive, précise, exhaustive, parfaitement adaptée aux machines.

Mais — et c'est l'enseignement du Chapitre 1 qui s'applique directement ici — les tests de régression automatisés ne valent que si ce qu'ils couvrent est réellement important. Une suite de régression qui couvre 2 000 scénarios sans intérêt laissera passer des bugs critiques aussi sûrement qu'une suite qui ne couvre que 100 scénarios bien choisis.

La règle d'or de la régression automatisée : **couvrir d'abord les risques, pas les fonctionnalités.** Un scénario automatisé devrait toujours pouvoir répondre à la question : *quel risque est-ce que j'adresse ?* Si la réponse est vague ou absente, le scénario n'a probablement pas sa place dans la suite prioritaire.

Le tableau de la valeur en FORGE

Activité automatisée	Ce qu'elle libère	Vers quoi
Tests unitaires (SMALL)	Vérification manuelle du comportement atomique	Confiance dans le refactoring et l'innovation
Intégration continue	Détection manuelle des conflits d'intégration	Développement parallèle sans anxiété
Déploiement continu	Rituels manuels et nocturnes de déploiement	Bien-être de l'équipe, réduction du risque release

Activité automatisée	Ce qu'elle libère	Vers quoi
Analyse statique	Détection manuelle d'erreurs syntaxiques/sécurité	Revues de code focalisées sur la logique métier
Tests de régression	Vérification manuelle répétitive post-modification	Temps pour le test exploratoire et la stratégie

2.3 Ce qui ne doit pas être automatisé en SENSE

La zone SENSE — qui comprend les phases ÉLICITER et DÉFINIR — est la zone où l'humain est le plus irremplaçable. Et c'est peut-être pour cette raison que c'est aussi la zone où la tentation d'automatiser est la plus séduisante — et la plus dangereuse.

L'élicitation des besoins : l'entretien humain est irremplaçable

L'élicitation des besoins n'est pas un processus de collecte d'informations. C'est un processus de *création de sens*. Quand un analyste ou un PO s'assoit avec un utilisateur pour comprendre ses besoins, ce qui se passe n'est pas un simple transfert d'informations d'un cerveau à un autre. C'est une co-construction de compréhension.

L'utilisateur ne sait pas toujours exactement ce qu'il veut. Il sait ce qu'il vit, ce qui le frustre, ce qui lui manque. Et c'est dans le dialogue — dans les questions, les reformulations, les silences, les hésitations, les exemples concrets tirés d'une expérience récente — que le besoin réel émerge. Souvent différent de ce que l'utilisateur aurait dit si on lui avait simplement demandé de remplir un formulaire.

Un outil automatisé peut collecter des réponses à des questions prédéfinies. Il ne peut pas percevoir la hésitation dans la voix de quelqu'un qui dit « oui, ça me conviendrait » alors que son visage dit quelque chose de très différent. Il ne peut pas improviser la question de suivi qui n'était pas prévue mais qui révèle soudainement le vrai problème sous-jacent. Il ne peut pas créer la confiance qui permet à l'utilisateur de partager une frustration profonde qu'il n'aurait jamais confié à un formulaire.

Les outils numériques — sondages, analytics comportementaux, enregistrements de sessions — peuvent *assister* l'élicitation. Ils peuvent fournir des données complémentaires, identifier des tendances, révéler des comportements que les

entretiens n'auraient pas captés. Mais ils ne peuvent pas *remplacer* l'entretien humain. Ils sont des inputs pour l'analyste — pas des substituts.

La définition des spécifications : la décision reste humaine

La rédaction des spécifications IF…THEN… est un processus de décision, pas seulement de documentation. Décider ce que le système doit faire dans un cas donné — et ce qu'il ne doit pas faire — implique un jugement sur ce qui est important, sur ce qui est faisable, sur ce qui est acceptable pour les utilisateurs et les parties prenantes.

Des outils peuvent suggérer des spécifications à partir de patterns existants. Des systèmes d'IA peuvent proposer des cas de test à partir d'une analyse sémantique des user stories. Ces assistances sont précieuses — elles accélèrent le travail et réduisent les omissions.

Mais la décision finale de ce que le produit fait ou ne fait pas reste irréductiblement humaine. Elle engage la responsabilité. Elle reflète des valeurs. Elle implique des compromis que seul un humain pleinement informé du contexte peut assumer.

La priorisation : les algorithmes suggèrent, les humains décident

Les algorithmes de priorisation peuvent analyser des centaines de variables — valeur commerciale estimée, effort de développement, dépendances techniques, feedbacks utilisateurs, tendances du marché — et proposer un classement du backlog. Cette analyse est précieuse parce qu'elle est plus exhaustive et moins biaisée que ce qu'un humain peut faire seul.

Mais la priorisation finale intègre des dimensions qu'aucun algorithme ne peut pleinement capturer : la vision à long terme du produit, les dynamiques politiques internes, la capacité réelle de l'équipe à livrer dans les délais, les signaux faibles du marché que seul quelqu'un immergé dans l'écosystème peut percevoir, et la responsabilité morale de tenir les engagements pris envers les utilisateurs.

La priorisation est une décision de leadership. Elle peut être informée par des outils automatisés. Elle ne peut pas être déléguée à eux.

2.4 Ce qui ne doit pas être automatisé en CARE

La zone CARE — qui comprend les phases DIFFUSER et CONFIRMER — est la zone où l'humain touche le produit à ses deux extrémités les plus vulnérables : la relation avec l'utilisateur et le jugement sur la qualité globale. Ces deux activités requièrent une présence humaine pleine et consciente.

La relation utilisateur : la confiance se construit humain à humain

Quand un utilisateur découvre une nouvelle fonctionnalité et qu'il est perdu ou frustré, ce dont il a besoin n'est pas un chatbot ou un guide interactif — même si ces outils peuvent aider. Ce dont il a besoin, c'est de sentir que quelqu'un comprend sa frustration, que quelqu'un est responsable du produit qu'il utilise, et que quelqu'un va travailler à améliorer son expérience.

Cette confiance — cette relation fondamentale entre un produit et ses utilisateurs — se construit dans des interactions humaines. Elle peut être soutenue par des outils automatisés. Elle ne peut pas être construite par eux. Et quand elle est endommagée — par un bug critique, par une interface incompréhensible, par une communication maladroite — seule une réponse humaine authentique peut la reconstruire.

La règle de CARE : les outils automatisés gèrent les volumes. Les humains gèrent les relations. Les deux sont nécessaires. Les deux doivent rester dans leurs zones respectives.

La décision de Go/No-Go : les métriques recommandent, l'humain décide

La décision de lancer — ou de ne pas lancer — une release en production est l'une des décisions les plus importantes dans le SDLC réconcilié. Elle engage la qualité du produit, la réputation de l'équipe et la confiance des utilisateurs.

Des outils automatisés peuvent assembler toutes les métriques pertinentes : taux de réussite des tests, couverture de code, résultats de l'analyse de performance, alertes de sécurité, disponibilité des systèmes dépendants. Cette assemblage automatisée

est précieuse — elle rend la décision plus informée et moins dépendante de la mémoire individuelle.

Mais la décision elle-même est humaine. Elle intègre des éléments que les métriques ne capturent pas : la confiance intuitive de l'équipe dans la release, les préoccupations soulevées lors de la session de test exploratoire, les signaux politiques ou commerciaux qui rendent un lancement plus ou moins risqué à un moment donné. Un humain peut dire non à une release que toutes les métriques décrivent comme safe — et avoir raison de le faire.

La cérémonie de qualimétrie : données automatiques, interprétation humaine

La cérémonie de qualimétrie — ce moment régulier où l'équipe examine ensemble les métriques de qualité de son SDLC — est, dans le SDLC réconcilié, une activité profondément humaine enveloppée dans des données automatisées.

Les données sont automatisées : tableaux de bord, tendances, anomalies statistiques. Leur assemblage est une tâche pour les machines. Mais l'interprétation — comprendre pourquoi le taux de bugs critiques a augmenté ce sprint, décider si cette tendance est préoccupante ou circonstancielle, identifier les actions correctives et distribuer les responsabilités — est une activité de jugement collectif qui ne peut pas être déléguée.

Une qualimétrie entièrement automatisée produirait des rapports que personne ne lirait vraiment, et des alertes que personne ne comprendrait vraiment. C'est précisément pour éviter cela que le SDLC réconcilié insiste sur la dimension cérémonielle de la qualimétrie : ce n'est pas juste un rapport. C'est une réunion. Une conversation. Un moment de responsabilité collective.

2.5 Le test exploratoire — ce qui ne sera jamais automatisé

Il existe une activité dans le SDLC réconcilié qui est, par sa nature même, *fondamentalement non automatisable*. Non pas parce que nous n'avons pas encore

les outils pour l'automatiser — mais parce qu'automatiser cette activité reviendrait à en détruire l'essence.

Cette activité est le test exploratoire.

Le test exploratoire est la capacité d'un testeur humain à explorer un produit de manière non scénarisée, guidé par sa curiosité, son intuition, son expertise du domaine et sa compréhension des utilisateurs. Un testeur exploratoire ne suit pas un script. Il navigue. Il questionne. Il observe. Il crée des situations que personne n'avait prévues. Il joue le rôle de l'utilisateur naïf, de l'utilisateur malveillant, de l'utilisateur pressé, de l'utilisateur senior qui connaît le produit par cœur et qui fait des choses étranges que personne d'autre ne fait.

Un robot peut exécuter un script de test. Un robot peut vérifier que le résultat d'une exécution correspond à un résultat attendu. Mais un robot ne peut pas explorer — parce que l'exploration nécessite une intention, une curiosité, une compréhension contextuelle et une capacité d'improvisation que les machines n'ont pas.

Ce point est fondamental. Il mérite d'être dit sans ambiguïté : **le test exploratoire ne sera jamais entièrement automatisé. Pas parce que c'est impossible techniquement. Parce que, si c'était possible, ce ne serait plus du test exploratoire.**

Des outils peuvent assister le testeur exploratoire : enregistrement des sessions, analyse des chemins de navigation, génération de données de test aléatoires. Mais le testeur lui-même — avec son intelligence, son expérience et sa curiosité — est et restera la composante irremplaçable de cette activité.

Dans le cadre du SDLC réconcilié, le test exploratoire est une priorité de la zone FORGE, une activité qui doit être activement protégée de la pression d'automatisation, et une compétence qui doit être cultivée et développée — pas érodée par une dépendance croissante aux suites de tests automatisés.

2.6 Anecdote terrain — La grande entreprise technologique spécialisée en systèmes embarqués temps réel

Anecdote terrain · Grande entreprise technologique spécialisée en systèmes embarqués temps réel

Les noms, dates, projets et données spécifiques ont été rendus anonymes conformément aux règles de la Collection ToTAL.

Dans une grande entreprise technologique spécialisée en systèmes embarqués temps réel, l'équipe chargée du développement de la plateforme de simulation avait construit, sur plusieurs années, un pipeline CI/CD d'une sophistication remarquable. Build automatisé. Tests de régression automatisés sur plusieurs dizaines de configurations matérielles. Déploiement automatisé dans les environnements de validation. Monitoring en temps réel des performances de simulation. Rollback automatique en cas de dégradation détectée.

Le tout fonctionnait avec une fiabilité impressionnante. Les releases étaient fréquentes. Le taux de succès des déploiements était très élevé. L'équipe était fière — à juste titre — de son infrastructure.

Puis vint le projet que nous appellerons la Mission Faucon. Un projet de simulation critique pour un client dans le secteur aérospatial. Les exigences étaient précises : la plateforme devait simuler, en temps réel et avec une précision définie contractuellement, le comportement d'un système de contrôle embarqué sous des conditions spécifiques de charge et de latence.

Le pipeline automatisé avait été étendu pour couvrir les nouvelles fonctionnalités de la Mission Faucon. Les tests de régression avaient été mis à jour. Le pipeline tournait. Tout semblait en ordre.

Mais il y avait un bug. Un bug subtil, contextuel, qui n'apparaissait que dans une combinaison très spécifique de conditions : lorsque la simulation était initialisée dans un certain mode de démarrage chaud (*warm start*), avec un profil de charge particulier, après une séquence précise d'opérations de calibration. Ce bug produisait une dérive progressive de la précision de simulation — une dérive qui restait dans les tolérances pendant les premières minutes, puis dépassait le seuil contractuel après environ huit minutes de simulation continue.

Les tests automatisés n'avaient jamais simulé exactement cette séquence. Pourquoi ? Parce que cette séquence n'avait pas été identifiée comme un scénario de test lors de la conception du pipeline. Elle correspondait à un cas d'usage opérationnel réel — un mode de démarrage utilisé par les ingénieurs clients dans des conditions de test spécifiques — mais pas à un scénario de régression standard.

C'est une testeure humaine, lors d'une session d'exploration non planifiée — elle avait simplement du temps libre un après-midi et avait décidé d'explorer le comportement de la plateforme dans des conditions inhabituelles — qui avait découvert la dérive. Elle avait reproduit les conditions, documenté ses observations, et alerté l'équipe. Le bug avait été corrigé avant la livraison.

La leçon de cet épisode n'est pas que le pipeline automatisé était mauvais. C'était un excellent pipeline. La leçon est que l'excellence du pipeline automatisé n'était pas suffisante à elle seule. La testeure humaine — avec sa curiosité, son intuition et sa liberté d'explorer — avait trouvé ce que le pipeline ne pouvait pas trouver.

Cette session de test exploratoire non planifiée a probablement évité un incident contractuel majeur. Son coût : quelques heures d'une ingénieure compétente qui avait du temps pour explorer. Son retour sur investissement : incalculable.

2.7 Scène DREAM-TEAM — La carte d'automatisation d'ORIGINALIUM

Scène DREAM-TEAM · ORIGINALIUM

Équipe : Rania (Tech Lead) · Théo (PO) · Karim (QA Lead) · ORIGINALIUM

Deux semaines après la réunion du Chapitre 1. ORIGINALIUM traverse une phase de croissance accélérée. L'équipe a doublé en six mois. Le produit principal est en train d'être refactorisé pour accommoder une architecture multi-tenant. La pression de livraison est réelle.

Karim a passé la semaine précédente à construire ce qu'il appelle « la carte d'automatisation d'ORIGINALIUM ». Sur le grand écran de la salle de réunion, il projette un tableau à trois colonnes : AUTOMATISER / ASSISTER / HUMAIN UNIQUEMENT. Sous chaque colonne, une liste d'activités du SDLC réconcilié.

Rania parcourt le tableau en silence pendant une minute. Puis : « La colonne ASSISTER est plus longue que les deux autres combinées. »

« Je sais, » dit Karim. « Soixante pour cent des activités que j'ai analysées tombent dans cette zone. Ni entièrement automatisables, ni entièrement manuelles. Ce sont des activités où on a besoin de données automatisées et de jugement humain, dans cet ordre. »

Théo se penche en avant. « C'est intéressant. Parce que c'est précisément la zone où on investit le moins. On investit dans l'automatisation pure — les pipelines, les tests automatisés — et on investit dans les activités humaines pures — les revues de code, les entretiens utilisateurs. Mais la zone ASSISTER… je ne suis pas sûr qu'on y pense vraiment. »

« C'est le point aveugle, » dit Rania. « La zone ASSISTER est la plus difficile parce qu'elle exige qu'on comprenne vraiment ce que l'outil automatise et ce que l'humain fait avec les résultats. Si l'humain ne sait pas interpréter les résultats que l'outil produit, l'outil n'a aucune valeur. »

Karim hoche la tête. « Et c'est exactement ce qu'on voit dans la qualimétrie. On a des tableaux de bord magnifiques. Mais lors des cérémonies, personne ne sait vraiment

quoi faire avec les tendances qui émergent. Les données sont là. L'interprétation n'est pas développée. »

La discussion qui suit dure une heure. Elle produit trois décisions concrètes.

La première : nommer explicitement, pour chaque outil automatisé dans la stack d'ORIGINALIUM, les compétences humaines nécessaires pour en tirer de la valeur. Si ces compétences ne sont pas présentes dans l'équipe, l'outil est une dette — pas un avantage.

La deuxième : réserver, dans chaque sprint, du temps explicitement protégé pour le test exploratoire. Pas un budget théorique — un slot dans le planning, avec un testeur nommé et un périmètre défini.

La troisième : organiser une session de formation sur l'interprétation des métriques de qualimétrie. Pas sur la lecture des graphiques — sur la signification des tendances, sur les corrélations entre les métriques, sur les décisions que ces tendances devraient déclencher.

La carte d'automatisation d'ORIGINALIUM est devenue un artefact de référence pour l'équipe. Elle est révisée tous les trimestres. Et la colonne ASSISTER — cette zone intermédiaire que personne ne regarde — est devenue la plus discutée de toutes.

À retenir — Chapitre 2

- Dans le SDLC réconcilié, les 6 phases se répartissent en trois postures vis-à-vis de l'automatisation : AUTOMATISER PRIORITAIREMENT (IMPLÉMENTER en FORGE), HUMAIN + ASSISTANCE (DÉFINIR, PLANIFIER, CONFIRMER), et HUMAIN UNIQUEMENT (ÉLICITER, DIFFUSER, relations utilisateurs).
- La zone ASSISTER — ni entièrement automatisable ni entièrement manuelle — représente environ 60% des activités du SDLC. C'est la zone la plus importante et la moins bien adressée dans la plupart des organisations.
- En FORGE, les tests SMALL, la CI, la CD, l'analyse statique et les tests de

régression sont des automatisations prioritaires qui libèrent la capacité humaine vers la réflexion architecturale et le test exploratoire.

- En SENSE, l'élicitation des besoins est irremplaçablement humaine. Les outils peuvent assister ; ils ne peuvent pas substituer le dialogue et la co-construction de compréhension.
- En CARE, les métriques peuvent être automatisées ; la décision de Go/No-Go, l'interprétation de la qualimétrie et la relation utilisateur restent irréductiblement humaines.
- Le test exploratoire ne sera jamais automatisé. Il est l'activité la plus irréductiblement humaine du SDLC. Il doit être protégé, budgété et cultivé.

Symptômes à observer

- Le test exploratoire n'est jamais planifié — il est « ce qu'on fait si on a du temps » à la fin du sprint. (Il n'est donc jamais fait.)
- Les cérémonies de qualimétrie ressemblent à des revues de tableaux de bord plutôt qu'à des décisions d'équipe.
- La décision de Go/No-Go est déléguée au pipeline — « si les tests passent, on déploie » — sans jugement humain explicite.
- Les besoins sont collectés via des formulaires ou des analytics, sans entretiens humains structurés avec les utilisateurs réels.
- Personne dans l'équipe ne peut nommer clairement quelles compétences humaines sont nécessaires pour exploiter les outils automatisés en place.

Questions de réflexion

1. Si vous deviez construire la carte d'automatisation de votre organisation — avec les colonnes AUTOMATISER / ASSISTER / HUMAIN UNIQUEMENT — où se situerait la majorité de vos activités actuelles ? Et où devrait-elle idéalement se situer ?
2. Dans votre équipe, la décision de Go/No-Go est-elle réellement humaine ? Ou est-elle, dans les faits, déléguée au pipeline automatisé ?
3. Combien de temps par sprint votre équipe consacre-t-elle explicitement au test exploratoire ? Est-ce planifié ou circonstanciel ?
4. Pouvez-vous nommer, pour chacun des outils automatisés de votre stack, les compétences humaines nécessaires pour en tirer de la valeur ? Si non, que révèle cette incapacité ?
5. La zone ASSISTER de votre organisation — les activités qui nécessitent à la fois des données automatisées et un jugement humain — est-elle bien adressée ? Qui est responsable de s'assurer que les humains savent interpréter les données que les machines produisent ?

Signal de vigilance

Si votre organisation investit massivement dans l'automatisation de FORGE tout en laissant s'appauvrir les activités humaines de SENSE et de CARE, le déséquilibre deviendra tôt ou tard visible — dans la qualité des besoins capturés, dans la pauvreté des décisions de release, et dans l'érosion de la relation de confiance avec les utilisateurs.

Un SDLC réconcilié n'est pas un pipeline entièrement automatisé. C'est un écosystème où l'automatisation et le jugement humain se renforcent mutuellement — chacun dans sa zone de pertinence maximale.

CHAPITRE 3

Les 4 pièges de l'automatisation mal ciblée

Comment des automatisations techniquement solides peuvent produire des résultats inverses à leurs intentions

Les chapitres précédents ont posé les bases : pourquoi on automatise, et où. Ce chapitre s'intéresse à ce qui arrive quand ces deux questions n'ont pas été correctement répondues. Non pas par incompétence — les équipes qui tombent dans ces pièges sont généralement compétentes et bien intentionnées — mais par une série de glissements progressifs, chacun rationnel en lui-même, qui conduisent collectivement à des résultats problématiques.

Ces pièges ont des noms. Ils se reconnaissent. Et ils se préviennent — à condition de savoir les chercher.

3.1 Piège 1 : Automatiser le mauvais problème

Piège 1

Automatiser le mauvais problème

« Si votre processus est mauvais, automatiser le rendra plus rapidement mauvais. »

C'est peut-être le plus fondamental des quatre pièges, parce qu'il est le seul qui soit irréparable sans revenir en arrière. Les trois autres pièges peuvent être corrigés progressivement, avec des ajustements et de la patience. Le premier piège — automatiser le mauvais problème — nécessite souvent une remise en question complète de l'automatisation en place.

La logique du mauvais problème

Automatiser le mauvais problème se produit quand on identifie un symptôme et qu'on automatise sa gestion, plutôt que d'identifier la cause sous-jacente et de la traiter. C'est l'équivalent logiciel d'un médecin qui prescrit un analgésique à un patient qui se plaint de maux de tête chroniques, sans jamais chercher à comprendre pourquoi les maux de tête surviennent.

L'analgésique peut fonctionner. Le patient peut se sentir mieux. Mais si la cause des maux de tête est une tension artérielle non contrôlée ou une tumeur nasale, traiter le symptôme est au mieux insuffisant, au pire dangereux.

Exemple 1 : Automatiser un rapport basé sur des métriques mal définies

Une équipe observe que les parties prenantes se plaignent de ne pas avoir de visibilité sur la qualité du produit. La décision est prise d'automatiser un tableau de bord de qualité, mis à jour en temps réel, accessible à toutes les parties prenantes.

Le tableau de bord est construit. Il est beau, réactif, riche en métriques. Taux de couverture de test, nombre de tests qui passent, densité de bugs détectés, temps moyen de résolution. Les parties prenantes ont maintenant une visibilité en temps réel.

Mais trois mois plus tard, les parties prenantes se plaignent toujours de ne pas comprendre l'état de la qualité du produit. Le tableau de bord est là. Les métriques sont automatisées. Et pourtant, la plainte persiste.

Analyse : le vrai problème n'était pas l'absence de données — c'était l'absence de compréhension partagée de ce que « qualité » signifie pour ce produit, dans ce contexte, pour ces parties prenantes. Les métriques automatisées répondent à cette

question... mais avec les mauvaises métriques. Parce que personne n'a pris le temps de définir, avec les parties prenantes, ce qu'elles voulaient vraiment savoir.

Automatiser la mauvaise mesure, plus rapidement, ne produit pas la bonne information. Elle produit un volume plus grand de mauvaise information.

Exemple 2 : Automatiser des tests qui ne testent pas les vrais risques

L'éditeur de logiciel ERP comptable du Chapitre 1 en est l'illustration parfaite. Mais ce problème est si répandu qu'il mérite d'être développé davantage.

La pression de couverture de test est réelle dans les équipes modernes. Les métriques de couverture — pourcentage de code couvert, nombre de tests automatisés — sont faciles à mesurer, faciles à présenter, et faciles à utiliser comme indicateur de maturité. Cette facilité crée une incitation perverse : augmenter la couverture en automatisant des tests faciles à écrire, même s'ils ne couvrent pas les risques réels du produit.

Un test qui vérifie qu'un bouton s'affiche en bleu quand il devrait s'afficher en bleu est un test facile à automatiser. Il contribue à la couverture. Et il est, dans la très grande majorité des cas, un test qui ne détectera jamais aucun bug réel — parce que les risques qui comptent pour les utilisateurs ne portent pas sur la couleur des boutons.

La règle du SDLC réconcilié : avant d'automatiser un test, nommer explicitement le risque que ce test adresse. Si le risque ne peut pas être nommé clairement, le test n'a probablement pas sa place dans la suite prioritaire.

Comment détecter qu'on automatise le mauvais problème

Trois questions diagnostiques :

- **Est-ce que l'automatisation résoudrait le problème si elle fonctionnait parfaitement ?** Si la réponse n'est pas clairement oui, on automatise probablement un symptôme plutôt que la cause.
- **Est-ce que quelqu'un peut nommer le problème que cette automatisation résout, en une phrase sans jargon technique ?** Si la

réponse demande un paragraphe de contexte technique, le lien entre l'automatisation et le problème n'est peut-être pas si direct.

- **Si on supprimait cette automatisation demain, quel problème reviendrait immédiatement ?** La réponse à cette question révèle ce que l'automatisation résout vraiment.

3.2 Piège 2 : La dette d'automatisation

Piège 2

La dette d'automatisation

Scripts mal maintenus, tests obsolètes, pipelines trop complexes : une forme de dette qui est souvent pire que la dette technique parce qu'elle est invisible.

La dette technique est un concept bien connu : le coût futur engendré par des décisions techniques sous-optimales prises dans le présent. Elle se manifeste en mauvais designs, en couplage excessif, en absence de documentation, en code difficile à maintenir.

La dette d'automatisation est son équivalent moins discuté — et souvent plus pernicieux. C'est le coût futur engendré par des automatisations qui ne sont pas maintenues, pas comprises, ou qui ont perdu leur pertinence mais continuent d'exister.

Les trois formes de la dette d'automatisation

Forme 1 : Les scripts mal maintenus. Toute automatisation est du code. Et comme tout code, elle peut vieillir mal. Les scripts de test écrits pour une version du produit peuvent devenir des boulets après une refonte majeure. Les scripts de déploiement configurés pour une ancienne architecture peuvent échouer de manière cryptique sur la nouvelle. Les scripts de monitoring calibrés pour un certain volume de trafic peuvent produire des faux positifs incessants quand le produit a grandi.

Le problème n'est pas que ces scripts vieillissent — c'est inévitable. Le problème est que personne ne les maintient activement, parce qu'ils « fonctionnent » — dans le sens où ils s'exécutent sans crash — même s'ils ne produisent plus rien d'utile.

Forme 2 : Les tests automatisés qui n'ont plus de sens. Après une refonte significative d'une fonctionnalité, les tests automatisés qui couvraient l'ancienne version du comportement doivent être mis à jour ou supprimés. Si personne ne le fait, on se retrouve avec des tests qui passent — parce que le code de l'ancienne fonctionnalité a été supprimé — mais qui ne testent plus rien du tout. Ou pire, des tests qui échouent systématiquement parce que la fonctionnalité a changé, et qui ont été mis en « ignore » par quelqu'un pressé, et qui sont restés là.

Une suite de tests avec 30% de tests en « ignore » n'est pas une suite de tests à 70% de couverture. C'est une suite de tests à 70% de couverture dont on ne sait pas si les 70% actifs testent encore ce qu'ils sont censés tester.

Forme 3 : Les pipelines trop complexes que personne ne comprend. C'est peut-être la forme la plus insidieuse de la dette d'automatisation. Un pipeline CI/CD construit progressivement, par accumulation de contributions individuelles, peut atteindre un niveau de complexité où plus personne dans l'équipe ne peut décrire précisément ce qu'il fait, pourquoi chaque étape existe, et comment le modifier en toute sécurité.

Ce pipeline continue à fonctionner — jusqu'au jour où il ne fonctionne plus. Et ce jour-là, l'équipe découvre qu'elle ne sait pas comment le réparer parce que personne ne l'a jamais vraiment compris.

Pourquoi la dette d'automatisation est souvent pire que la dette technique

La dette technique est visible. Elle se manifeste dans la lenteur du code, dans la fréquence des bugs, dans la difficulté à ajouter de nouvelles fonctionnalités. Un développeur qui travaille sur du code mal maintenu le ressent immédiatement.

La dette d'automatisation, elle, est invisible. Un test automatisé qui ne teste plus rien continue de passer. Un pipeline trop complexe continue de s'exécuter. Un tableau de bord automatisé qui mesure les mauvaises métriques continue de s'afficher.

Cette invisibilité est dangereuse parce qu'elle crée un sentiment de sécurité infondé. L'équipe croit que tout va bien — les tests passent, le pipeline tourne, les tableaux de bord sont verts — alors que la réalité est que ces automatisations ne protègent plus la qualité du produit de la manière dont elles devraient.

La règle de remboursement de la dette d'automatisation

Dans le SDLC réconcilié, la maintenance des automatisations est une activité de première classe, pas une activité secondaire. Elle doit être explicitement planifiée, budgétée et mesurée. La règle est simple : pour chaque sprint où l'équipe investit dans de nouvelles automatisations, elle doit également consacrer du temps à la revue et à la maintenance des automatisations existantes.

Un audit trimestriel de la suite de tests automatisés — identifiant les tests obsolètes, les tests en « ignore », les tests qui ne testent plus rien de pertinent — est une pratique minimale. Un audit annuel du pipeline CI/CD global, avec documentation de chaque étape et de sa raison d'être, est une pratique avancée.

3.3 Piège 3 : L'atrophie des compétences

Piège 3

L'atrophie des compétences

Quand une compétence est entièrement automatisée, la capacité humaine à l'exercer s'atrophie — souvent de manière irréversible à court terme.

Ce piège est le plus philosophiquement important des quatre. Il touche directement à la question centrale de L13 : quel est le rôle de l'humain dans un monde de plus en plus automatisé ? Et comment s'assurer qu'il reste au centre ?

L'exemple des pilotes d'avion

Dans l'aviation, la question de l'atrophie des compétences est débattue depuis des décennies. Les avions modernes sont capables de décoller, naviguer et atterrir de manière entièrement automatisée. Les pilotes supervisent le système automatique.

Dans les conditions normales d'opération, ils interagissent peu avec le pilotage manuel.

La conséquence, documentée dans de nombreuses études et enquêtes d'accidents, est que les compétences de pilotage manuel se dégradent avec le temps. Des pilotes qui ont passé des années à superviser des systèmes automatiques peuvent, dans une situation d'urgence qui désactive les automatismes, se retrouver en difficulté face à un avion qu'ils doivent piloter manuellement.

Ce n'est pas une hypothèse théorique. C'est une réalité documentée qui a conduit les autorités de l'aviation civile à exiger des heures minimales de pilotage manuel pour maintenir les compétences des pilotes — même dans les opérations entièrement automatisées.

L'équivalent logiciel : l'équipe qui ne sait plus tester sans son outil

Dans l'industrie du logiciel, l'atrophie des compétences se manifeste de manière moins dramatique mais tout aussi réelle.

L'équipe QA qui a entièrement délégué ses tests de régression à un outil automatisé peut progressivement perdre la compétence de concevoir des cas de test pertinents. Elle peut perdre la connaissance profonde du domaine métier qui permet d'identifier les scénarios à risque. Elle peut perdre la capacité de réflexion stratégique sur la couverture de test — parce que cette réflexion est devenue, de facto, une responsabilité de l'outil.

Quand l'outil tombe en panne — ou quand il doit être remplacé, ou quand l'équipe doit tester un nouveau type de fonctionnalité que l'outil ne sait pas gérer — cette atrophie devient visible. Et souvent douloureuse.

L'équipe qui ne sait plus déployer sans son pipeline

Un autre exemple, peut-être moins évident : l'équipe DevOps qui a entièrement délégué ses déploiements à un pipeline automatisé peut perdre la compétence de déployer manuellement. Pas parce que le déploiement manuel est compliqué — il ne l'est généralement pas, si on le comprend — mais parce qu'en l'absence de pratique

régulière, les procédures manuelles ne sont plus mémorisées, les points de contrôle ne sont plus réflexes, et les décisions à prendre dans un déploiement manuel complexe ne sont plus instinctives.

Dans une situation d'urgence — incident de production, pipeline indisponible, déploiement critique nécessaire dans les 30 minutes — cette atrophie peut avoir des conséquences sérieuses.

La règle ToTAL : on n'automatise jamais entièrement une compétence qu'on veut maintenir

Cette règle est l'une des plus importantes du SDLC réconcilié, et elle mérite d'être formulée sans ambiguïté.

> *On n'automatise jamais entièrement une compétence qu'on veut conserver dans l'organisation.*

Si une compétence doit être maintenue — parce qu'elle est irremplaçable dans certaines situations d'urgence, parce qu'elle est fondamentale à la compréhension du produit, parce qu'elle est nécessaire pour former les nouvelles recrues — alors elle doit être régulièrement exercée, même quand l'outil automatisé la rend superflue dans les opérations courantes.

Cela implique :

- Des exercices réguliers de déploiement manuel, même quand le pipeline fonctionne parfaitement.
- Des sessions de conception de cas de test sans outils automatisés, même quand un framework d'automatisation est en place.
- Des sessions d'analyse manuelle des logs, même quand un système d'analyse automatisée est opérationnel.
- Des revues de code approfondies, même quand l'analyse statique automatisée couvre la majorité des problèmes syntaxiques.

Ces exercices peuvent sembler redondants. Ils ne le sont pas. Ils sont la garantie que l'organisation conserve sa capacité d'action dans les situations où l'automatisation ne suffit pas.

L'atrophie organisationnelle

Il existe enfin une forme d'atrophie particulièrement pernicieuse : l'atrophie organisationnelle. Elle survient quand une compétence est tellement automatisée que *personne dans l'organisation ne la détient plus*. Pas parce que les individus l'ont oubliée — mais parce que les individus qui la détenaient ont été réaffectés, sont partis, ou ont pris leur retraite, et que leurs successeurs n'ont jamais eu à l'apprendre parce que l'automatisation s'en chargeait.

L'organisation se retrouve alors dans une situation paradoxale : elle dispose d'automatisations sophistiquées dont elle dépend entièrement, sans avoir en son sein les compétences nécessaires pour les comprendre, les maintenir, ou les remplacer si nécessaire.

La solution à l'atrophie organisationnelle est structurelle : la documentation des compétences implicites, le transfert de savoir explicite entre générations de praticiens, et l'intégration dans les programmes de développement professionnel de la pratique régulière des compétences fondamentales — même celles qui sont largement automatisées.

3.4 Piège 4 : La fausse sécurité

Piège 4

La fausse sécurité

100% de tests qui passent ≠ 100% de qualité. Les tests automatisés testent ce qu'on a décidé de tester — pas ce qui pourrait réellement échouer.

Ce piège est peut-être le plus subtil — et certainement le plus répandu. Il exploite la psychologie du confort statistique : quand les chiffres sont bons, on se sent bien. Et

dans l'industrie du logiciel, les chiffres les plus visibles sont souvent les résultats des tests automatisés.

La logique de la fausse sécurité

La fausse sécurité n'est pas le résultat de mauvaises intentions. Elle est le résultat d'une confusion entre deux propositions qui ressemblent à la même chose mais qui sont fondamentalement différentes :

Proposition A : « Tous nos tests automatisés passent. »

Proposition B : « Notre produit est de haute qualité. »

La Proposition A implique la Proposition B *si et seulement si* les tests automatisés sont conçus pour couvrir les risques réels du produit avec une précision suffisante. Dans la réalité, cette condition n'est que rarement pleinement satisfaite.

Les tests automatisés testent ce qu'on a *décidé* de tester. Cette décision a été prise à un moment donné, par des personnes qui avaient une compréhension partielle du produit, de ses utilisateurs et de ses risques. La réalité du produit a évolué depuis. Les comportements des utilisateurs ont évolué. Les intégrations avec des systèmes externes ont évolué. Les attentes qualité ont évolué. Les tests, eux, ont peut-être moins évolué.

Ce que les tests automatisés ne peuvent pas trouver

Les tests automatisés, aussi bien conçus soient-ils, ne peuvent pas trouver :

- **Les bugs dans les scénarios non-prévus :** par définition, un test automatisé teste un scénario prévu. Le bug le plus dangereux est souvent celui qui survient dans la combinaison de conditions que personne n'avait imaginée.
- **Les problèmes d'expérience utilisateur :** un test automatisé peut vérifier qu'un formulaire soumet correctement les données. Il ne peut pas détecter que le formulaire est confus, que les libellés sont ambigus, ou que l'ordre des champs est contre-intuitif.

- **Les régressions subtiles dans des contextes non-testés :** un bug qui n'apparaît que sur une configuration matérielle spécifique, dans une locale particulière, après une séquence précise d'opérations, ne sera trouvé par les tests automatisés que si quelqu'un a eu la présence d'esprit de tester exactement cette configuration, cette locale et cette séquence.
- **Les problèmes de performance sous des charges réelles non-simulées :** les tests de performance automatisés simulent un comportement de charge. La charge réelle des utilisateurs en production est souvent différente — et parfois dramatiquement différente — de ce qui a été simulé.
- **Les problèmes qui émergent de l'interaction entre plusieurs systèmes :** dans un environnement distribué, les bugs les plus difficiles émergent de l'interaction entre systèmes dans des états particuliers. Ces interactions sont difficiles à reproduire et encore plus difficiles à automatiser.

L'antidote à la fausse sécurité

L'antidote à la fausse sécurité n'est pas d'arrêter d'automatiser les tests. C'est de maintenir une lucidité constante sur ce que les tests automatisés couvrent — et sur ce qu'ils ne couvrent pas.

Concrètement :

- Tenir une liste explicite des risques non-couverts par l'automatisation — et s'assurer qu'ils sont adressés par d'autres mécanismes, notamment le test exploratoire.
- Ne jamais utiliser le taux de réussite des tests automatisés comme seul indicateur de qualité dans la décision de Go/No-Go.
- Maintenir des sessions régulières de test exploratoire comme complément systématique aux tests automatisés.
- Réviser régulièrement la pertinence de la suite de tests automatisés par rapport à l'évolution du produit et de ses risques.

> *Les tests automatisés sont une assurance contre les régressions connues. Le test exploratoire est une assurance contre les risques inconnus. L'un sans l'autre est incomplet.*

3.5 Anecdote terrain — Le studio de développement de jeux vidéo

Anecdote terrain · Studio de développement de jeux vidéo

Les noms, dates, projets et données spécifiques ont été rendus anonymes conformément aux règles de la Collection ToTAL.

Un studio de développement de jeux vidéo, reconnu dans l'industrie pour la qualité de ses productions, avait investi massivement dans l'automatisation de son processus QA. Au moment des faits, la stack d'automatisation comprenait plus de 12 000 tests automatisés, un système de déploiement continu sur plusieurs plateformes cibles, et un monitoring complet des métriques de performance en temps réel sur des versions de build représentatives.

L'infrastructure était impressionnante. Elle avait été construite progressivement, avec soin, par une équipe technique compétente. Lors des revues internes, elle était présentée comme un avantage compétitif significatif : des cycles de validation accélérés, une détection précoce des régressions, une capacité de déploiement multi-plateforme sans paralysie opérationnelle.

Le studio lançait un DLC (contenu téléchargeable) majeur pour l'un de ses titres phares. Le DLC avait traversé l'ensemble du processus de validation automatisée. Les 12 000 tests passaient. Les métriques de performance étaient dans les seuils. Le monitoring ne signalait rien d'anormal. Le Go/No-Go avait été donné avec confiance.

La release eut lieu. Dans les premières heures, des milliers de plaintes commencèrent à affluer sur les plateformes communautaires. Un bug critique rendait le DLC injouable pour un sous-ensemble significatif de joueurs : ceux qui possédaient également un autre DLC publié plusieurs mois auparavant, et qui avaient atteint un certain stade de progression dans le jeu de base.

Le bug n'était pas dans le code du nouveau DLC seul. Il était dans l'interaction entre le nouveau DLC, l'ancien DLC et un état de progression spécifique du joueur. Une combinaison que, techniquement, les testeurs auraient pu reproduire — mais que personne n'avait pensé à inclure dans les scénarios de test.

La raison ? La construction des scénarios de test automatisés avait été guidée par la logique du module : tester le nouveau DLC de manière isolée, tester son intégration avec le jeu de base dans ses états standards. L'interaction avec l'ancien DLC, dans des états de progression avancés, était une combinaison que personne n'avait identifiée comme un scénario de risque.

L'analyse post-incident fut sans appel : un testeur exploratoire en session libre — quelqu'un qui joue vraiment au jeu, avec tous les DLC installés, dans une progression avancée — aurait probablement trouvé ce bug. Parce que c'est exactement le type de joueur qui aurait découvert le DLC le premier jour, dans les conditions les plus exigeantes.

Les 12 000 tests automatisés testaient le jeu tel qu'on l'avait imaginé. Le testeur exploratoire aurait testé le jeu tel que les joueurs passionnés le vivent.

Ce n'est pas la même chose.

La leçon de cet épisode touche simultanément au Piège 1 (les tests couvraient l'architecture modulaire, pas les risques d'interaction réels), au Piège 3 (personne dans l'équipe ne pratiquait régulièrement les jeux avec tous les DLC installés — une compétence de testeur en conditions réelles qui s'était atrophiée) et au Piège 4 (la fausse sécurité des 12 000 tests au vert avait éliminé le signal d'alarme qui aurait dû pousser à plus d'exploration).

Dans les mois qui suivirent l'incident, le studio introduisit un programme de test exploratoire structuré, obligatoire pour chaque DLC, effectué par des testeurs qui jouaient régulièrement aux jeux dans des conditions de joueur avancé. L'infrastructure automatisée fut conservée — elle était précieuse pour la détection des régressions. Mais elle ne fut plus jamais la seule source de confiance dans la qualité.

À retenir — Chapitre 3

- **Piège 1 — Automatiser le mauvais problème :** automatiser un symptôme plutôt que sa cause produit plus rapidement les mêmes mauvais résultats. Identifier la vraie cause avant d'automatiser la solution.
- **Piège 2 — La dette d'automatisation :** les automatisations vieillissent. Les scripts mal maintenus, les tests obsolètes et les pipelines incompréhensibles créent une dette souvent plus dangereuse que la dette technique — parce qu'elle est invisible.
- **Piège 3 — L'atrophie des compétences :** on n'automatise jamais entièrement une compétence qu'on veut maintenir. L'exercice régulier des compétences fondamentales — même quand l'automatisation les rend superflues au quotidien — est une nécessité organisationnelle.
- **Piège 4 — La fausse sécurité :** des tests qui passent ne signifient pas un produit de qualité. Les tests automatisés testent les risques connus. Le test exploratoire est irremplaçable pour les risques inconnus.
- Ces quatre pièges peuvent survenir simultanément — et s'amplifier mutuellement. La vigilance doit être systémique, pas ponctuelle.

Symptômes à observer

- Des bugs critiques sont découverts en production alors que tous les tests automatisés passent — de manière récurrente.
- Personne dans l'équipe ne peut décrire précisément ce que fait le pipeline CI/CD à chaque étape, et pourquoi.
- La suite de tests contient des tests en « ignore » ou des tests qui échouent de manière intermittente et dont la cause n'est pas investiguée.
- En cas de panne du pipeline ou d'un outil automatisé, l'équipe est paralysée plutôt que de basculer sur des alternatives manuelles.
- La couverture de test est présentée comme un indicateur de qualité sans contextualisation des risques couverts.
- Les nouvelles recrues ne savent pas comment effectuer manuellement les tâches que les outils automatisent — et personne n'a jamais pensé à les leur enseigner.

Questions de réflexion

1. Dans votre suite de tests automatisés actuelle, combien de tests couvriraient des risques réels identifiés par les parties prenantes — et combien testent des comportements nominaux peu susceptibles de causer des problèmes en production ?
2. Si votre pipeline CI/CD tombait en panne demain matin, combien de temps faudrait-il pour que l'équipe reprenne une opération normale par des moyens alternatifs ? Ce délai vous semble-t-il acceptable ?
3. Pouvez-vous nommer des compétences qui existaient dans votre équipe il y a trois ans et qui se sont atrophiées parce qu'un outil les a progressivement remplacées ?
4. Lors de votre dernière décision de Go/No-Go, quelle part de la confiance

reposait sur les résultats des tests automatisés, et quelle part sur d'autres sources de signal (test exploratoire, revue humaine, intuition de l'équipe) ?

5. Y a-t-il dans votre stack des automatisations dont vous ne savez plus vraiment ce qu'elles font — et que vous avez peur de supprimer parce qu'il pourrait se passer quelque chose ?

Signal de vigilance

Si votre organisation a subi un bug critique en production que vos tests automatisés n'ont pas détecté, et si la réaction à cet incident a été « ajouter plus de tests automatisés » sans reconfigurer la stratégie de test, vous êtes probablement en train de creuser le Piège 4 plus profondément.

La bonne réaction à un bug critique manqué est de comprendre *pourquoi* l'automatisation n'a pas détecté ce bug, de remédier à la cause racine (lacune de couverture, mauvais scénarios, absence de test exploratoire), et seulement ensuite d'ajuster les automatisations en conséquence.

CHAPITRE 4

ONETOOL : la tentation de l'outil miracle

Comment la promesse d'un outil qui tout résout engendre une dépendance qui fragilise l'organisation

Chaque génération technologique produit son ONETOOL. Dans les années 1990, c'était le progiciel ERP intégré qui promettait de gérer l'ensemble des processus d'entreprise dans une seule interface. Dans les années 2000, c'était la plateforme de tests automatisés qui promettait de couvrir tous les niveaux et tous les types de tests. Dans les années 2010, c'était l'outil DevOps intégré qui promettait d'unifier le développement, la livraison et l'opération dans un pipeline cohérent. Dans les années 2020, c'est l'IA générative qui promet de remplacer les ingénieurs humains dans la rédaction du code, des tests, de la documentation et de la gestion des incidents.

ONETOOL n'est pas un outil spécifique. C'est un *archétype*. L'archétype de l'outil qui promet de tout résoudre — ou du moins d'éliminer les parties les plus difficiles du travail de développement logiciel. Et cet archétype réapparaît, sous des formes différentes, avec une régularité troublante.

4.1 L'histoire de ONETOOL

L'histoire se passe dans une entreprise SaaS en hypercroissance. Apperons-la ce qu'elle est : une organisation qui grandit plus vite qu'elle ne peut gérer sa propre complexité. Les équipes s'étoffent. Les produits se multiplient. Les processus qui fonctionnaient à 20 personnes ne fonctionnent plus à 80.

Dans ce contexte de tension, un vendeur arrive avec ONETOOL. ONETOOL est présenté comme la solution à la fragmentation : un seul outil pour gérer le cycle de développement de bout en bout, de la gestion des user stories au monitoring de production. Des démonstrations éblouissantes. Des références clients impressionnantes. Une intégration avec les outils existants promise et documentée.

Phase 1 : L'euphorie (mois 1 à 4)

Les premiers mois sont exactement ce que le vendeur avait promis. ONETOOL s'installe bien, l'onboarding est fluide, les intégrations fonctionnent. L'équipe est enthousiasmée par la promesse d'unification. Les tableaux de bord de ONETOOL sont beaux. Les rapports automatiques sont précis. La vision d'une plateforme unifiée semble se concrétiser.

L'enthousiasme est réel. Et il n'est pas entièrement injustifié. ONETOOL apporte des bénéfices tangibles dans les premiers mois : une meilleure visibilité sur l'état global des projets, une réduction de la friction entre des outils qui ne se parlaient pas, une expérience utilisateur plus cohérente pour les membres de l'équipe.

Mais, pendant cette période d'euphorie, quelque chose se passe en silence. Les anciens outils — certains maladroits, d'autres simplement différents de ONETOOL — sont progressivement abandonnés. Les configurations spécifiques que l'organisation avait construites pour ses cas d'usage particuliers sont laissées de côté au profit des workflows standard de ONETOOL. Les équipes qui avaient leurs propres pratiques et leurs propres rituels s'alignent sur les pratiques imposées par la structure de ONETOOL.

Phase 2 : La normalisation silencieuse (mois 5 à 12)

Un an après l'adoption de ONETOOL, quelque chose de plus insidieux est en cours : l'organisation s'est progressivement restructurée autour de ONETOOL plutôt que l'inverse. Les processus qui ne s'intégraient pas bien dans ONETOOL ont été abandonnés ou modifiés pour s'y adapter. Les pratiques que ONETOOL ne supportait pas ont été oubliées.

Ce phénomène est normal — dans une certaine mesure. Tout changement d'outil implique une adaptation des pratiques. Le problème survient quand l'adaptation est à sens unique : quand c'est toujours l'organisation qui s'adapte à l'outil, jamais l'outil qui s'adapte à l'organisation.

À ce stade, la dépendance à ONETOOL est déjà significative — mais pas encore visible. Elle le deviendra.

Phase 3 : La révélation de la dépendance (à partir de l'année 2)

La révélation de la dépendance survient généralement à l'occasion d'un incident technique. ONETOOL a un problème. Une panne partielle, une migration de version difficile, une incompatibilité avec un nouveau système. Et soudain, l'organisation réalise ce qu'elle a perdu : sa capacité d'opérer sans ONETOOL.

Les alertes sont en place dans ONETOOL — mais plus personne ne sait comment surveiller les métriques critiques sans le tableau de bord de ONETOOL. Les workflows de code review passent par ONETOOL — mais les pratiques de revue de code qui existaient avant ONETOOL sont oubliées. Les rapports de qualimétrie sont générés par ONETOOL — mais personne ne sait quelles métriques prioriser si les rapports automatiques ne sont pas disponibles.

L'organisation est paralysée. Non pas parce qu'elle est incompétente, mais parce qu'elle a progressivement externalisé des compétences organisationnelles fondamentales à un outil unique. Et cet outil, maintenant, n'est plus disponible.

Phase 4 : La reconstruction (mois 18 à 36)

La sortie de ONETOOL est longue, coûteuse et douloureuse. Elle oblige l'organisation à reconstituer des compétences perdues, à reconstructer des pratiques oubliées et à identifier les fonctionnalités de ONETOOL dont elle avait vraiment besoin — pour les remplacer par des alternatives plus légères et moins centralisées.

Le coût de sortie de ONETOOL est presque toujours supérieur au coût d'adoption. Et c'est précisément ce coût asymétrique qui rend la décision initiale d'adoption si difficile à évaluer correctement.

4.2 La loi de la dépendance à l'outil

L'histoire de ONETOOL illustre ce qu'on peut appeler la loi de la dépendance à l'outil dans le SDLC réconcilié :

> *Plus un outil est puissant et intégré, plus la dépendance qu'il crée est grande — et plus le coût de sortie est élevé.*

Cette loi n'est pas un argument contre les outils puissants. C'est un argument pour une adoption délibérée et informée. Elle implique trois responsabilités pour toute organisation qui envisage un outil d'automatisation significatif.

Responsabilité 1 : Comprendre ce que l'outil fait

Pas à un niveau d'abstraction marketing — mais à un niveau opérationnel suffisant pour exercer un jugement sur sa valeur. Qu'est-ce que l'outil automatise exactement ? Quels sont ses algorithmes de décision ? Quelles données consomme-t-il, et que produit-il ? Comment ses résultats doivent-ils être interprétés ?

Si l'équipe ne peut pas répondre à ces questions, l'outil n'est pas encore prêt à être adopté. Non pas parce que l'outil est mauvais — mais parce que l'équipe n'est pas prête à en extraire de la valeur responsablement.

Responsabilité 2 : Maintenir la capacité à opérer sans l'outil

Cette responsabilité découle directement du Piège 3 (atrophie des compétences). Si l'organisation adopte un outil pour automatiser une compétence, elle doit simultanément planifier comment maintenir cette compétence en l'absence de l'outil.

Cela ne signifie pas refuser l'outil. Cela signifie intégrer dans les pratiques organisationnelles des exercices réguliers — peut-être trimestriels, peut-être annuels selon l'outil et la criticité de la compétence — où les équipes opèrent sans l'outil pendant une période limitée. Ces exercices maintiennent la compétence. Ils révèlent également les zones où la dépendance est la plus forte — et permettent d'agir avant qu'une situation d'urgence ne les révèle à un coût plus élevé.

Responsabilité 3 : Avoir un plan de sortie viable

Avant d'adopter tout outil d'automatisation significatif, l'organisation devrait pouvoir répondre à la question : *si nous devions remplacer cet outil dans 18 mois, comment le ferions-nous ?*

Cette question n'est pas un exercice de pessimisme. C'est un exercice de gouvernance. Si la réponse est *« ce serait impossible »* ou *« ça prendrait des années et nous paralyserait »*, alors l'adoption de l'outil crée une dépendance stratégique qui devrait être explicitement discutée et acceptée en connaissance de cause — pas subie par défaut.

4.3 Comment évaluer un outil d'automatisation

Le SDLC réconcilié propose un cadre d'évaluation structuré pour tout outil d'automatisation significatif. Ce cadre s'articule autour de cinq questions, posées dans l'ordre.

Question 1 : Qu'est-ce qu'il automatise exactement ?

La réponse doit être précise et opérationnelle. Non pas *« il automatise le pipeline CI/CD »*, mais *« il automatise l'exécution des tests sur chaque commit, le build de l'artefact déployable, le déploiement en environnement de staging, et la notification des résultats à l'équipe. Il ne couvre pas la décision de promotion vers la production. »*

Cette précision permet de distinguer ce que l'outil fait de ce qu'il ne fait pas — et donc de ne pas tomber dans la fausse sécurité de croire qu'il couvre plus qu'il ne le fait.

Question 2 : Quelles compétences atrophie-t-il ?

Pour chaque activité qu'il automatise, identifier la compétence humaine correspondante qui risque de s'atrophier, et planifier comment cette compétence sera maintenue.

Question 3 : Quel est le coût de sortie ?

Pas seulement le coût financier de la licence. Le coût de sortie inclut : le coût de migration des données et des configurations, le coût de reconstitution des pratiques qui s'étaient adaptées à l'outil, et le coût humain de la transition — la friction organisationnelle, la période d'apprentissage, la productivité réduite pendant la migration.

Un outil avec un faible coût de sortie peut être adopté avec plus de liberté. Un outil avec un coût de sortie élevé requiert une justification stratégique explicite et une gouvernance de long terme.

Question 4 : Comprend-on ce qu'il fait à l'intérieur ?

Les boîtes noires sont dangereuses. Un outil dont on ne comprend pas les décisions internes — un algorithme de priorisation automatique, un système d'analyse de code

par IA, un outil de génération de cas de test — crée une dépendance cognitive : l'équipe ne peut pas exercer de jugement sur les outputs de l'outil, parce qu'elle ne comprend pas comment ces outputs ont été produits.

Cette question ne disqualifie pas les outils complexes. Elle exige que leur adoption s'accompagne d'un investissement dans la compréhension de leurs mécanismes — au niveau nécessaire pour exercer un jugement raisonné sur leurs outputs.

Question 5 : Si cet outil était indisponible pendant une semaine, que se passerait-il ?

C'est la question diagnostique ultime. Elle révèle, avec une précision remarquable, le niveau réel de dépendance de l'organisation à cet outil. Une organisation résiliente peut répondre : *« On ralentirait, on utiliserait des alternatives manuelles ou partielles, et on s'en sortirait. »* Une organisation fragile répond : *« Ce serait une catastrophe, on ne sait pas comment travailler sans lui. »*

La deuxième réponse n'est pas nécessairement disqualifiante — certains outils sont si fondamentaux que leur indisponibilité prolongée est effectivement une situation d'urgence majeure. Mais dans ce cas, cela doit être su, nommé et géré en conséquence : haute disponibilité contractuelle, plans de continuité, redondances.

La grille d'évaluation ONETOOL

Question	Réponse idéale	Signal d'alarme
Qu'est-ce qu'il automatise exactement ?	Réponse précise et opérationnelle, avec limites explicites	Réponse vague : « Il automatise plein de choses »
Quelles compétences atrophie-t-il ?	Liste identifiée, plan de maintien défini	Question non posée, compétences non identifiées
Quel est le coût de sortie ?	Estimé, documenté, jugé acceptable	Inconnu ou « ce serait impossible »
Comprend-on ce qu'il fait à l'intérieur ?	Niveau de compréhension suffisant pour exercer un jugement	« On lui fait confiance » sans compréhension des mécanismes

Question	Réponse idéale	Signal d'alarme
Indisponible une semaine : que se passe-t-il ?	On ralentit, on a des alternatives, on s'en sort	Paralysie complète de l'organisation

4.4 Scène DREAM-TEAM — La tentation ONETOOL

Scène DREAM-TEAM · ORIGINALIUM

Équipe : Rania (Tech Lead) · Théo (PO) · Karim (QA Lead) · ORIGINALIUM

Un mardi après-midi. Dans la petite salle de conférence d'ORIGINALIUM, un représentant commercial d'une entreprise spécialisée en automatisation de la qualité logicielle présente sa solution. Appelons-la ONETOOL. La présentation est soignée, les démos sont fluides, les références clients sont rassurantes. La promesse est claire : ONETOOL gère l'ensemble du cycle de test automatisé, de la génération des cas de test à partir des spécifications jusqu'à la génération automatique des rapports de qualimétrie, en passant par l'exécution, l'analyse des résultats et la priorisation des bugs.

Rania écoute en prenant des notes. Théo pose des questions sur l'intégration avec le backlog et le workflow de priorisation. Le représentant répond avec aisance. La démonstration se déroule bien.

Karim est silencieux depuis le début. À la 40e minute, il lève la main.

« J'ai cinq questions. »

Le représentant sourit. « Bien sûr. »

« Première question : qu'est-ce que ONETOOL automatise exactement ? Pas en termes marketing — en termes opérationnels. Qu'est-ce qu'il fait à chaque étape, et qu'est-ce qu'il ne fait pas ? »

Le représentant répond bien à cette première question. Karim prend des notes.

« Deuxième question : si mon équipe utilise ONETOOL pour générer ses cas de test depuis deux ans, et que ONETOOL n'est plus disponible, est-ce que mon équipe sait encore concevoir des cas de test elle-même ? Avez-vous des clients qui ont traversé ça ? »

Légère hésitation du représentant. Une réponse partielle. Karim prend note de l'hésitation.

« Troisième question : si ONETOOL est indisponible pendant une semaine complète — panne, problème de licence, n'importe quoi — est-ce qu'on peut continuer à travailler ? »

Silence de deux secondes. Puis : « Il faudrait voir avec votre équipe comment vous configurez les alternatives… »

Karim pose son stylo. Doucement. Il regarde Rania, puis Théo. Ils se comprennent sans parler.

Karim se retourne vers le représentant et dit, avec beaucoup de calme : « La réponse à cette question me dit à peu près tout ce que j'ai besoin de savoir. »

Ce n'est pas la fin de la conversation avec ONETOOL. Ce n'est pas un rejet définitif. C'est le début d'une évaluation beaucoup plus rigoureuse, guidée par les cinq questions du cadre. L'équipe d'ORIGINALIUM demandera une preuve de concept de trois mois, avec maintien parallèle des pratiques manuelles existantes. Elle nommera explicitement les compétences qu'elle veut conserver. Elle exigera une clause contractuelle de portabilité des données et des configurations.

Peut-être que ONETOOL sera adopté. Peut-être pas. Mais dans un cas comme dans l'autre, la décision sera délibérée, informée et réversible. Pas subie.

C'est ça, la gouvernance de l'automatisation responsable.

À retenir — Chapitre 4

- ONETOOL est un archétype — pas un outil spécifique. Chaque génération

technologique produit sa version de l'outil miracle qui promet de tout résoudre. La vigilance doit être constante.

- L'adoption de ONETOOL suit un cycle prévisible : euphorie → normalisation silencieuse → révélation de la dépendance → reconstruction douloureuse. Reconnaître ce cycle avant d'entrer dans la phase 3 est l'objectif.
- La loi de la dépendance : plus un outil est puissant et intégré, plus la dépendance qu'il crée est grande et plus le coût de sortie est élevé. Ce coût asymétrique doit être explicitement évalué avant l'adoption.
- Les trois responsabilités avant adoption : comprendre ce que l'outil fait, maintenir la capacité à opérer sans lui, avoir un plan de sortie viable.
- La cinquième question de Karim — « si ONETOOL est indisponible une semaine, peut-on continuer à travailler ? » — est la question diagnostique la plus simple et la plus révélatrice de toutes.
- L'évaluation d'un outil n'est pas un événement ponctuel. C'est un processus continu, qui se réexamine à chaque renouvellement de contrat et à chaque évolution significative de l'outil ou des besoins de l'organisation.

Symptômes à observer

- L'équipe ne peut pas nommer une activité importante que ONETOOL effectue, sans utiliser le jargon de ONETOOL pour la décrire.
- La migration vers une alternative à ONETOOL n'est jamais évaluée sérieusement parce qu'elle semble trop compliquée à imaginer.
- Les processus de l'organisation ont été progressivement modifiés pour s'adapter à ONETOOL, plutôt que l'inverse.
- En cas de bug ou de comportement inattendu de ONETOOL, l'équipe n'a pas la capacité de comprendre ce qui se passe — elle attend que le support

de ONETOOL réponde.

- La décision d'adopter ONETOOL a été prise sous le coup de l'enthousiasme (phase d'euphorie) sans évaluation structurée du coût de sortie.
- Les nouvelles recrues apprennent directement à utiliser ONETOOL — sans jamais apprendre les pratiques fondamentales que ONETOOL est censé supporter.

Questions de réflexion

1. Dans votre organisation, quel est votre ONETOOL actuel — l'outil dont la dépendance est la plus grande ? Et est-ce que cette dépendance est délibérée et assumée, ou est-elle le résultat d'une adoption progressive non planifiée ?
2. Si vous deviez remplacer votre outil le plus critique dans 18 mois, combien de temps et d'effort cela représenterait-il ? Ce coût de sortie est-il documenté quelque part dans votre organisation ?
3. Quelles compétences votre équipe a-t-elle perdues — ou n'a-t-elle jamais développées — parce qu'un outil s'en chargeait ?
4. La dernière fois que votre ONETOOL a eu un incident, quelle a été la réaction de l'équipe ? Était-elle capable d'opérer en mode dégradé, ou était-elle paralysée ?
5. Quand un vendeur vous présente un nouvel outil, est-ce que vous posez systématiquement les cinq questions du cadre d'évaluation ? Ou est-ce que la démonstration et les références clients suffisent généralement à convaincre ?

Signal de vigilance

Si votre organisation ne peut pas répondre honnêtement à la question « que ferions-nous si notre outil le plus critique était indisponible pendant deux semaines ? », c'est une urgence de gouvernance — indépendamment de la fiabilité actuelle de l'outil.

La résilience organisationnelle ne se construit pas dans l'urgence. Elle se construit délibérément, avant que la crise n'arrive. Et pour les organisations qui utilisent des outils d'automatisation complexes, cette résilience passe par la compréhension, la pratique régulière des alternatives, et le maintien d'un plan de sortie viable — même si ce plan n'est jamais utilisé.

Collection ToTAL — Réconciliation · Livre 13 · Chapitre 4 · Images et couverture par Soléane Bélanger

---

CONCLUSION — PARTIE A

L'automatisation comme acte humain délibéré

Ce que cette Partie A a construit — et ce que la Partie B va approfondir

---

Nous voici à la fin de la Partie A. Quatre chapitres qui ont cherché à répondre à une question que l'industrie du logiciel pose rarement avec l'honnêteté et la profondeur qu'elle mérite : pourquoi les humains automatisent — et ce que cette réponse révèle sur la manière dont ils devraient le faire.

Faisons le point sur ce que nous avons construit ensemble.

Ce que la Partie A a établi

Le **Chapitre 1** a établi que l'automatisation est une pulsion humaine ancestrale, aussi vieille que le moulin à eau et le métier à tisser Jacquard. Cette pulsion est animée par cinq motivations fondamentales : la libération, la précision, l'échelle, la sécurité et la curiosité. Ces cinq motivations sont toutes légitimes et toutes potentiellement piégeuses. La plus répandue dans l'industrie du logiciel — la curiosité pure — est aussi la plus dangereuse quand elle n'est pas questionnée. L'oncle MIDI l'illustre parfaitement : un ingénieur brillant qui a automatisé l'output musical sans jamais se demander si cet output était ce que sa famille valorisait vraiment dans la musique.

La question fondamentale que ce chapitre a posée — *Qu'est-ce que cette automatisation libère, et vers quoi ?* — n'est pas une question technique. C'est une question de gouvernance humaine. Et elle doit être posée et répondue avec précision avant tout investissement significatif dans l'automatisation.

Le **Chapitre 2** a ancré cette question dans la réalité du SDLC réconcilié. Il a proposé une cartographie des trois postures d'automatisation — AUTOMATISER PRIORITAIREMENT, HUMAIN + ASSISTANCE, HUMAIN UNIQUEMENT — et a montré que la majorité des activités du SDLC appartiennent à la zone intermédiaire ASSISTER, la plus importante et la moins bien adressée dans la plupart des organisations. Il a établi que certaines activités — l'élicitation des besoins, la relation utilisateur, la décision de Go/No-Go — sont irréductiblement humaines. Et il a défendu avec conviction le test exploratoire comme l'activité la plus fondamentalement non automatisable du cycle de développement.

Le **Chapitre 3** a nommé les quatre pièges classiques de l'automatisation mal ciblée : automatiser le mauvais problème, accumuler une dette d'automatisation invisible, laisser s'atrophier les compétences humaines essentielles, et développer une fausse sécurité statistique. Ces quatre pièges ne surviennent pas par incompétence — ils surviennent précisément parce que les équipes qui y tombent sont compétentes et bien intentionnées, mais ont laissé des glissements progressifs se cumuler sans les reconnaître.

Le **Chapitre 4** a introduit l'archétype de ONETOOL — l'outil miracle qui promet de tout résoudre — et a proposé un cadre d'évaluation rigoureux pour éviter les dépendances non délibérées. La question diagnostique de Karim — *si ONETOOL est indisponible une semaine, peut-on continuer à travailler ?* — est peut-être la plus simple et la plus révélatrice de toutes les questions qu'une organisation peut se poser sur sa maturité en matière de gouvernance de l'automatisation.

Le fil rouge de la Partie A

À travers ces quatre chapitres, un fil rouge court. Il peut être formulé ainsi :

> *L'automatisation est un acte humain délibéré — ou elle est un glissement inconscient. La différence entre les deux n'est pas technique. Elle est dans la question qu'on pose avant d'automatiser.*

Les organisations qui automatisent avec excellence ne sont pas celles qui ont les meilleurs ingénieurs ou les meilleurs outils. Ce sont celles qui, avant chaque investissement d'automatisation significatif, posent honnêtement la question fondamentale : *vers quoi est-ce que ça libère ?* Et qui ont le courage de ne pas automatiser quand la réponse n'est pas claire.

Cette discipline est rare. Elle est inconfortable, parce qu'elle impose une pause dans un secteur qui valorise la vitesse et l'action. Elle est invisible, parce que les meilleures décisions d'automatisation sont souvent celles qu'on ne prend pas — et qu'on ne peut donc pas montrer. Et elle est essentielle, parce qu'elle est la seule garantie que l'automatisation reste ce qu'elle a toujours voulu être : un outil au service de l'humain, et non l'inverse.

Ce que la Partie B va approfondir

La Partie A a posé les fondations philosophiques et analytiques. Elle a répondu à la question *pourquoi*. La Partie B va répondre à la question *comment*.

Comment concevoir une stratégie d'automatisation qui honore les principes établis dans la Partie A ? Comment construire une suite de tests automatisés qui couvre les vrais risques et non les scénarios faciles ? Comment maintenir les compétences humaines essentielles dans un environnement de plus en plus automatisé ? Comment intégrer le test exploratoire comme pratique de première classe aux côtés des tests automatisés ? Comment évaluer et choisir les outils d'automatisation avec la rigueur qu'ils méritent ?

La Partie B sera le complément opérationnel de cette Partie A philosophique. Elle apportera des frameworks, des outils de décision, des checklists et des exemples terrain détaillés. Elle s'appuiera sur les principes établis ici et les traduira en pratiques concrètes, applicables dès le prochain sprint.

Mais avant d'aller là, un moment d'arrêt s'impose. L'oncle MIDI n'a pas répondu à la question de sa nièce. Cette question — *c'est beau, mais c'est toi qui joues ?* — est restée suspendue dans l'air de la cave familiale. Peut-être qu'il n'avait pas de réponse. Peut-être que la réponse était trop inconfortable. Peut-être qu'il réalisa, à ce moment précis, qu'il voulait recommencer à jouer.

Ce livre ne dit pas que l'oncle MIDI a eu tort de construire son système MIDI. Il dit qu'il aurait été plus riche — plus heureux, plus connecté à ce qu'il aimait vraiment — s'il s'était posé la question fondamentale avant de commencer. Pas parce que la réponse aurait nécessairement été différente. Mais parce qu'une décision consciente vaut toujours plus qu'un glissement inconscient — même quand elles conduisent au même endroit.

Automatiser est un acte humain. Faisons-en un acte délibéré.

Synthèse de la Partie A — L'essentiel en 10 points

1. L'automatisation est une pulsion humaine ancestrale, noble dans son intention de libérer l'humain du répétitif.
2. Les 5 motivations profondes — Libération, Précision, Échelle, Sécurité,

Curiosité — sont toutes légitimes et toutes potentiellement piégeuses.

3. La curiosité pure (*je peux donc je dois*) est la motivation la plus répandue et la plus dangereuse quand elle n'est pas questionnée.

4. La question fondamentale : *Qu'est-ce que cette automatisation libère, et vers quoi ?* La réponse doit être précise et vérifiable.

5. Dans le SDLC réconcilié, environ 60% des activités appartiennent à la zone ASSISTER — ni entièrement automatisables ni entièrement manuelles.

6. L'élicitation des besoins, la relation utilisateur et le jugement de Go/No-Go sont irréductiblement humains.

7. Le test exploratoire ne sera jamais automatisé. Il doit être protégé, budgété et cultivé comme compétence de première classe.

8. Les 4 pièges — mauvais problème, dette d'automatisation, atrophie des compétences, fausse sécurité — peuvent survenir simultanément et s'amplifier mutuellement.

9. On n'automatise jamais entièrement une compétence qu'on veut maintenir.

10. Avant tout outil ONETOOL : comprendre ce qu'il fait, maintenir la capacité à opérer sans lui, avoir un plan de sortie viable.

Vers la Partie B — Trois questions pour y aller préparé

1. Dans votre organisation, si vous deviez aujourd'hui refaire votre carte d'automatisation (colonnes AUTOMATISER / ASSISTER / HUMAIN UNIQUEMENT), où seriez-vous ? Où voudriez-vous être ?

2. Quel est, dans votre contexte, le risque le plus important qui n'est actuellement adressé ni par vos automatisations ni par vos activités

humaines structurées ?

3. Si vous deviez nommer un seul principe de cette Partie A qui change concrètement quelque chose dans votre pratique dès demain matin, lequel serait-ce ?

La Partie B — La mécanique de l'automatisation responsable — est la suite naturelle de cette Partie A. Elle vous attend.

Collection ToTAL — Réconciliation · 21 livres

Livre 13 · Partie A — Pourquoi les humains automatisent : les origines profondes

Images et couverture par Soléane Bélanger

Léo Michel Bélanger, fondateur d'ORIGINALIUM